RA23502 – AUTONOMOUS MOBILE ROBOT

DESIGN AND SIMULATION OF INDUSTRY STANDARD AMRs (AGILEX SCOUT MINI)

Submitted By:

SAILAKSHMI B T (2024511019)

BACHELOR OF ENGINEERING
in
ROBOTICS AND AUTOMATION



DEPARTMENT OF PRODUCTION TECHNOLOGY
MADRAS INSTITUTE OF TECHNOLOGY
ANNA UNIVERSITY - CHENNAI 600044

BONAFIDE CERTIFICATE

Certified that this project report “Design and Simulation of Industry Standard AMRs (AgileX Scout Mini)” is the Bonafide work of **Sailakshmi B T (2024511019)** who carried out the project work under my supervision.

SIGNATURE

SUPERVISOR

TABLE OF CONTENTS:

S.No	TOPICS	PAGE
1	Introduction	5
2	Robot Specifications	5
3	Sensors Used and the Reason of Usage	6
4	SolidWorks Design	7
5	Conversion to URDF	8
6	Importing in MATLAB and Simulink	9
7	Building the Simscape Multibody Model	10
8	Creating the Warehouse	13
9	Autonomous Navigation	15
10	Wheel Slip and Traction Management	19
11	Algorithms Used	21
12	System Architecture	22
13	Additional MATLAB Visual Simulation (ISO 3691-4 Style)	25
14	How to Set It Up	28
15	Mathematical Formulation of the Implementation	31
16	Verification and Discussion of the Results	34
17	Limitations and Future Enhancements	35
18	Conclusion	36
19	References	36
A	Complete Parameter Listing	36
B	Glossary	37
C	Source Code of the Main Files	37

Abstract

This report describes the design and simulation of an industry-standard autonomous mobile robot (AMR), the AgileX Scout Mini four-wheel skid-steer platform, as a warehouse robot in MATLAB R2025b with Simulink and Simscape Multibody. The robot was modelled in SolidWorks (Assem2 with the parts chassis, chassis_top, W and up), exported to the Unified Robot Description Format (URDF) with STL meshes, and imported into Simulink with the Simscape Multibody function `smimport` and into the Robotics System Toolbox with `importrobot`. The imported model was turned into a working dynamic plant: the chassis is a free 6-DOF body, the four wheels are torque-driven revolute joints, and a custom tyre-ground contact model (spring-damper normal force with smoothed Coulomb friction) supplies the forces, including low-friction oil and water patches on the floor. A 3D warehouse (nine storage racks with goods, walls, pickup and delivery stations, floor boxes) is part of the multibody scene, together with four moving obstacles (two workers, a second AMR and a pallet truck). The robot is autonomous: a simulated 360° LiDAR feeds an occupancy grid, an A* planner finds and re-plans routes, a pure-pursuit controller follows them, and a mission state machine runs the sequence start, pickup, load, delivery, unload. A wheel-slip management layer (slip-ratio estimation, torque limiting, traction-control window and friction estimation) keeps the wheels from spinning or locking. In the reference run the robot completed the mission in 49.6 s over 32.0 m with 3 route corrections, 0 collisions and a minimum clearance of 0.22 m. The results are simulation only: the pose comes from the simulator (no localisation), LiDAR detection and collision checks are analytic computations on the warehouse geometry, the moving obstacles follow scripted trajectories, and no test was done on the physical robot. These limits are repeated in the relevant sections.

1. Introduction

An autonomous mobile robot (AMR) senses its surroundings, builds a representation of them, decides where to go and moves there without a fixed track or a human driver. In a warehouse an AMR carries goods between stations, and it must avoid racks, boxes, forklifts and people. Unlike an automated guided vehicle (AGV), which follows wires, tape or magnets, an AMR plans its own route and re-plans when the route is blocked. Testing such a robot on a real warehouse floor is slow and risky, so the software is first developed in simulation.

The robot used in this project is the AgileX Scout Mini, a compact four-wheel skid-steer chassis. The chassis and wheels were modelled by the author in SolidWorks (assembly Assem2) and converted to URDF. The project went through the following stages, which are also the structure of this report:

- SolidWorks CAD model of the robot, export to URDF and STL meshes (sections 4 and 5);
- import into MATLAB with `importrobot` and into Simulink with `smimport` (Simscape Multibody) (section 6);
- turning the import into a physical plant: free chassis, torque-driven wheels, tyre-ground friction (section 7);
- a 3D warehouse with racks, goods, stations, spills, static boxes and moving workers, robot and truck (section 8);
- autonomous navigation: LiDAR, occupancy grid, A* planning, re-planning, path following, mission logic (sections 9 and 11);
- wheel-slip detection and management, with a comparison of slip management on and off (section 10);
- an earlier MATLAB-script simulation of the same robot with ISO 3691-4 style safety fields (section 13);
- documentation of the code and the commands so that the work can be repeated (section 14).

Tool-chain: from the SolidWorks model to the simulation results

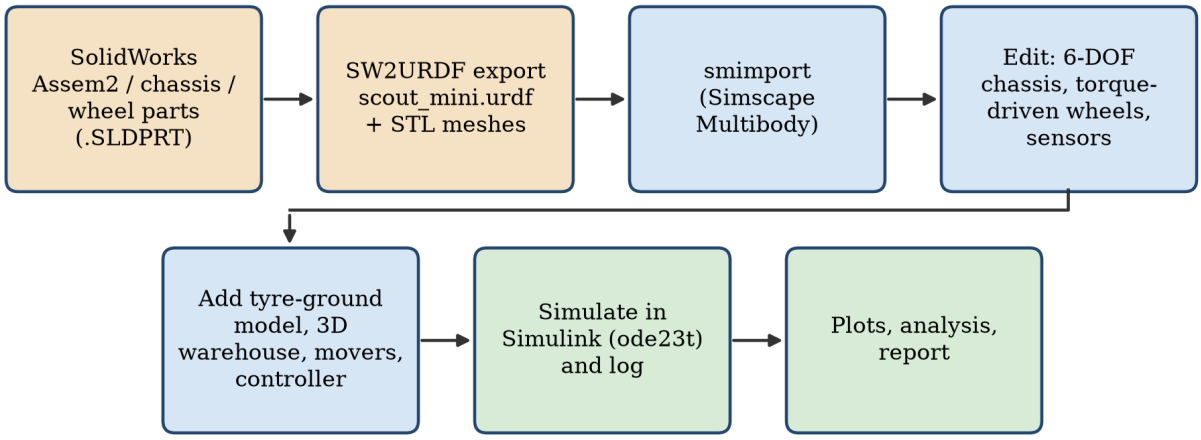


Fig. 1: From the SolidWorks model to the simulation results

The scope is simulation only. The wheel-ground forces, the wheel dynamics and the chassis motion are simulated by Simscape Multibody. The LiDAR, the obstacle geometry and the collision check are software models. Localisation error, battery and motor electrical models, the real Scout Mini firmware and hardware tests are not covered.

2. Robot Specifications

The AgileX Scout Mini is a four-wheel-drive skid-steer platform for research, education and light transport. The table lists the values of the model that was built and simulated in this project. They come from the author's SolidWorks

model, the exported URDF and the controller parameters; they are not measurements of the real robot, and the manufacturer's data sheet figures were not re-checked against them.

Parameter	Value in this project	Source
Type of drive	Four-wheel skid-steer, all wheels driven (hub-motor style torque on every wheel)	URDF, Simulink model
Links / joints	base_link + 4 wheels (w1…w4); 4 continuous wheel joints (j1…j4); fixed mounts for LiDAR, IMU, camera	`scout_mini.urdf`
Chassis mesh size	0.672 × 0.580 × 0.225 m (x, y, z in the base_link frame)	`base_link.STL` bounding box
Wheel mesh	Radius 0.090 m, width 0.124 m (largest extent of `w1.STL`); radius used for rolling in the controller and tyre model 0.085 m	w1…`w4.STL`, `scout_amr_params.m`
Track (left-right wheel distance)	0.612 m	Gazebo plug-in value in the URDF, controller
Wheel base (front-rear)	0.391 m	Joint origins in the URDF
Mass	Chassis 22.0 kg + 4 wheels × 1.5 kg + small sensor links ≈ 28.3 kg (URDF); 30 kg is used in the acceleration feed-forward	URDF, `scout_amr_params.m`
Wheel joint limits	Effort 40 N·m, velocity 25 rad/s (URDF); the controller limits motor torque to 17.5 N·m per wheel	URDF, `scout_amr_params.m`
Speed limit in the mission	$v \leq 1.2$ m/s, acceleration demand ≤ 2.5 m/s², yaw-rate demand slewed at 4 rad/s²	`scout_amr_params.m`, controller
Tyre-ground contact	Normal stiffness 12 000 N/m and damping 700 N·s/m per wheel; friction $\mu = 0.9$ on dry concrete, 0.15 on oil, 0.30 on water	`scout_amr_params.m`
Payload, battery, protection class	Not modelled	–

Applications. In this project the robot is used as a warehouse transport AMR: it drives from a start dock to a pickup station, waits while loading, drives through the aisles to a delivery station and waits while unloading. The same platform is used in research and education for navigation, mapping and control, and for indoor and outdoor patrolling.

3. Sensors Used and the Reason of Usage

An industrial AMR needs exteroceptive sensors (what is around the robot) and proprioceptive sensors (how the robot itself moves). Three groups of sensors appear in this project, and the table below states for each one whether it is part of the Simulink simulation or only declared in the robot description.

Sensor	Configuration	Reason for use	Status
2D LiDAR (Simulink simulation)	72 beams over 360° (5° spacing), maximum range 8 m, 10 Hz, exact ray–rectangle and ray–circle intersection with the warehouse geometry (`scout_wh_lidar.m`); no noise	Detects racks, boxes, workers, the second AMR and the truck; its hits build the dynamic layer of the occupancy grid, trigger re-planning and give the reactive repulsion and stop-before-contact limit	Simulated, used by the navigation
Body-state sensors (Transform Sensor blocks)	World-frame position, rotation matrix, linear and angular velocity of the base_link and of each wheel, sampled by the controller every 5 ms	Provide the robot pose and speed (stand-in for localisation and odometry) and the wheel positions for the tyre model	Simulated, ideal (true pose)
Wheel joint speed (encoder equivalent)	Joint velocity of each of the four wheel joints, 5 ms, no quantisation or noise	Closed-loop wheel-speed control and slip-ratio estimation	Simulated, ideal
Collision monitor	Ground-truth distance from the robot collision circle (r = 0.36 m) to every rack, wall, box, mover; counts overlaps	Measures safety of the run; it is not seen by the controller	Simulated (evaluation only)
2D LiDAR (URDF declaration)	Gazebo gpu_lidar: 720 samples over 360°, range 0.12–16 m, 20 Hz, Gaussian noise σ = 0.01 m	Navigation and SLAM in the earlier ROS 2 / Gazebo project that the URDF comes from	Mount imported as a body; scan model replaced by the analytic LiDAR above
IMU (URDF declaration)	Gazebo imu: 100 Hz, gyro noise σ = 0.0002 rad/s, accelerometer noise σ = 0.017 m/s²	Orientation and yaw-rate feedback for sensor fusion in ROS 2	Mount imported as a body; no IMU signal is simulated in Simulink
RGB-D camera (URDF declaration)	Gazebo rgbd_camera: 640 × 480 RGB, field of view 80°, depth 0.1–10 m	Forward perception (obstacle classification, depth)	Mount imported as a body; not simulated in Simulink
Wheel odometry (URDF declaration)	Gazebo DiffDrive plug-in: 50 Hz, wheel separation 0.612 m, wheel radius 0.08 m, limits 2.5 m/s and 3.5 rad/s	Dead-reckoning odometry in ROS 2	Not used in Simulink

Visual-simulation sensors (section 13)	LiDAR 64 beams, 8 m, σ = 0.015 m, 1 % dropouts; 6-DOF IMU with gyro bias 0.0025 rad/s; wheel odometry with 2 % drift	Illustrate sensor imperfections and ISO 3691-4 style protective fields	Simulated in the MATLAB script only

By type, the sensors of the project are:

- Exteroceptive (sense the environment): a 2D planar LiDAR (simulated in Simulink, with a 3D-camera mount and a second, noisy LiDAR model in the script simulation of section 13); an RGB-D camera is declared in the URDF only.
- Proprioceptive (sense the robot itself): wheel joint-speed sensors (encoder equivalent) on all four wheels, Transform Sensors for chassis pose and velocity (stand-in for localisation and odometry), and an IMU (gyroscope and accelerometer; declared in the URDF and simulated in the script simulation).
- Evaluation only: the collision monitor, which measures the ground-truth distance to obstacles and is not available to the controller.

A planar LiDAR is sufficient for the warehouse of this project because every obstacle is a vertical prism (rack, box, wall, worker cylinder, truck) that extends well above the scan plane, so one plane sees everything the robot could hit. The LiDAR in the Simulink model is an analytic model, not a measurement: it has no noise, no dropped returns and no minimum range; its ranges are only rounded to 2 cm. The robot pose used by the navigation is the true simulator pose, so no localisation, SLAM or sensor fusion is implemented in the Simulink project; the EKF, SLAM Toolbox and Nav2 configuration of the ROS 2 repository the URDF comes from were not run in this work.

4. SolidWorks Design

The robot was modelled by the author in SolidWorks. The files are in the repository Saibts/AgileX-scout-mini-sw (uploaded on 17 August 2026): the assembly `Assem2.SLDASM` and the parts `chassis.SLDPRT`, `chassis_top.SLDPRT`, `W.SLDPRT` (wheel) and `up.SLDPRT`. The assembly consists of one chassis body with a raised top cover carrying two rails, four wheels with hub spokes and the lettering "AGILE X" and "SCOUT MN" on the sides. The wheel axles are carried by four small brackets on the chassis sides. The URDF-export coordinate systems and axes (Origin_j1…, Axis_j1…) for the four wheel joints were created in the assembly for the exporter. Native SolidWorks files cannot be opened in MATLAB, so the geometry reaches MATLAB through the exported STL meshes (section 5).

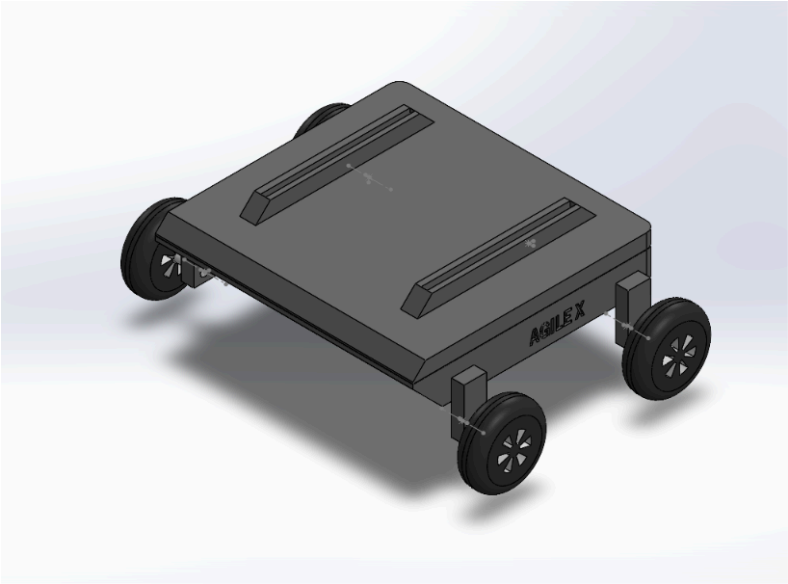


Fig. 2: Isometric view of the Scout Mini model in SolidWorks

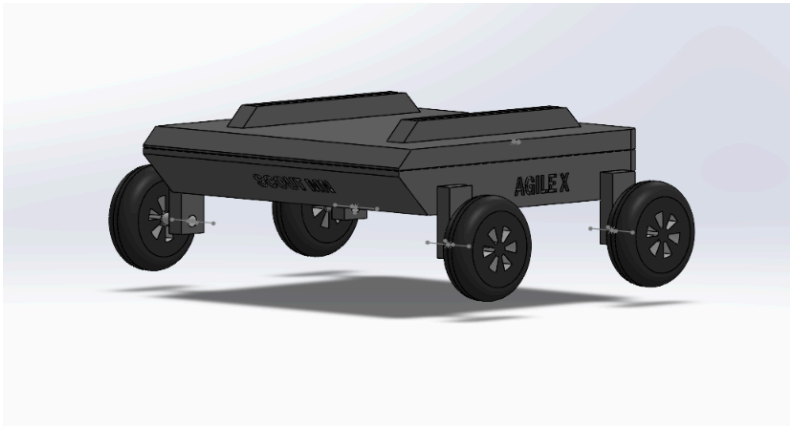


Fig. 3: Side view with the "AGILE X" and "SCOUT MN" lettering

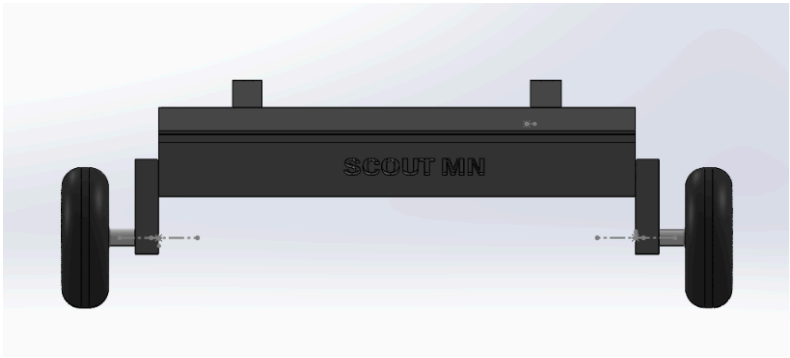


Fig. 4: Front view showing the wheel axles and brackets

The mass properties and the dimensions that are used by the simulation are the ones stored in the URDF and the STL files (section 5). How closely the CAD model matches the real Scout Mini was not verified against the real robot.

5. Conversion to URDF

The Unified Robot Description Format (URDF) is an XML file that describes a robot as a tree of links (rigid parts with mass, inertia, visual and collision shapes) and joints (how a child link is attached to its parent: type, origin and axis). An STL file holds only the triangle mesh of a shape; the URDF says where each mesh sits and how it moves. The meshes are in metres.

The assembly was exported with the SolidWorks-to-URDF exporter into the ROS 2 package assem2_robot of the author's repository AgileX_ScoutMini_JAZZY (urdf/`Assem2.SLDASM`.urdf and `meshes/base_link.STL`, `w1.STL`…`w4.STL`). That URDF was later edited by hand for the ROS 2 / Gazebo project: it carries analytic collision shapes (a box for the chassis, cylinders for the wheels), an adjusted low centre of mass, fixed mounts for the LiDAR, IMU and camera, and Gazebo plug-ins and sensor definitions. For the MATLAB work the file was copied to `scout_mini.urdf`, and the only change was to replace the ROS path `package://assem2_robot/meshes/` by the relative path `meshes/`, because MATLAB does not know the package:// convention.

Quantity	Value	How it was obtained
Links / joints	6 main links (base_footprint, base_link, w1…w4) plus the sensor links; 4 continuous wheel joints	`scout_mini.urdf`
Link masses	base_link 22.0 kg, wheels 1.5 kg each, LiDAR mount 0.08 kg + LiDAR 0.125 kg, IMU 0.01 kg, camera 0.04 + 0.07 kg	URDF inertial tags
Chassis inertia (about its centre of mass)	Ixx 0.35, Iyy 0.45, Izz 0.65 kg·m²	URDF
Wheel inertia	Ixx 0.005, Iyy = Izz 0.003 kg·m²	URDF
Wheel joint axes	Local x-axis (+1 for j1, j2; −1 for j3, j4); joint limit effort 40 N·m, velocity 25 rad/s	URDF
Wheel joint positions in base_link	(0.139, −0.297), (0.139, 0.094), (−0.473, −0.297), (−0.473, 0.093) m; z ≈ −0.146 m	URDF joint origins

Base_link in base_footprint	(−0.102, 0.167, 0.227) m, yaw +90°	URDF
Chassis mesh bounding box	x −0.503…0.169, y −0.415…0.165, z −0.169…0.056 m	`base_link.STL`
Wheel mesh	radius 0.090 m, width 0.124 m	`w1.STL`
Mesh triangles	base_link 1 628, each wheel 2 152	STL headers

The URDF contains base_footprint as a root link joined to base_link by a fixed joint. This fixed joint is the one that was replaced by a 6-DOF joint in the Simulink model (section 7).

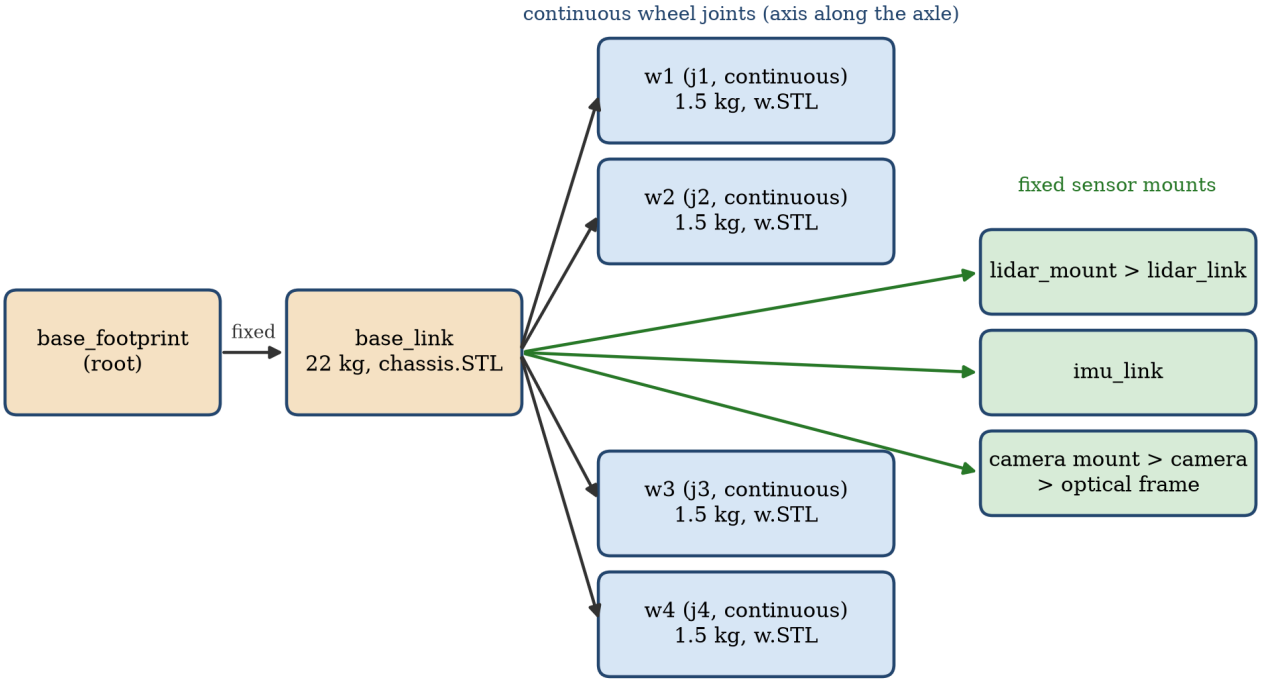


Fig. 5: Link and joint tree of scout_mini.urdf: base_footprint, base_link, four continuous wheel joints and the fixed sensor mounts

6. Importing in MATLAB and Simulink

The robot was imported in two ways. First, `importrobot` (Robotics System Toolbox) turns the URDF into a `rigidBodyTree`, which is used for visualisation and to read the wheel transforms. Second, `smimport` (Simscape Multibody) turns the same URDF into a Simulink model made of Simscape Multibody blocks: Solid blocks with the STL geometry, Rigid Transforms for the joint frames, Revolute Joints for the wheels and a Weld Joint for the base footprint.

6.1 Code and commands

```
>> cd 'D:\academic_shit\SEM5-III\AMR ROS\simulink_amr'
>> rbt = importrobot('scout_mini.urdf');       % rigidBodyTree
>> show(rbt, homeConfiguration(rbt), 'Visuals', 'on');
>> smimport('scout_mini.urdf');                % Simscape Multibody model (several minutes)
>> build_scout_amr_model                       % imports (or reuses the cached import), then
builds scout_amr_sim.slx
```

`smimport` takes several minutes for this model, so build_scout_amr_model saves the raw import as `scout_import_raw.slx` and reuses it on later builds. An important property of the imported model: the joint frames created by `smimport` have their Z axis along the wheel axle (not the X axis written in the URDF). The tyre model and the slip-ratio sign convention were built on this fact; using the wrong axis had put the wheel hubs up to 10 cm off and tilted the chassis at the start of the simulation.

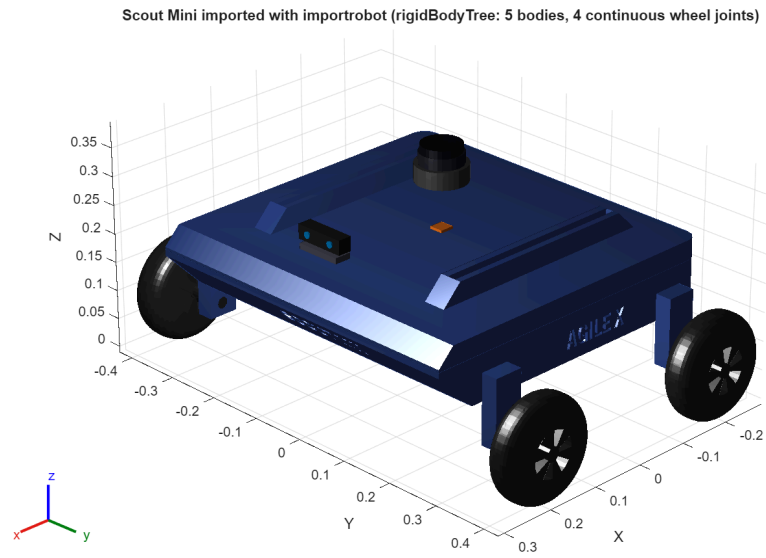


Fig. 6: The Scout Mini imported with importrobot (rigidBodyTree)

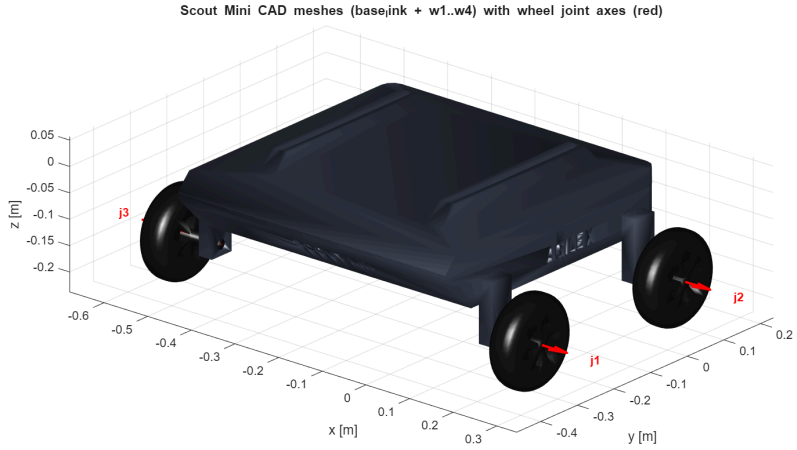


Fig. 7: The CAD meshes (base_link and w1…w4) placed with the URDF transforms; red arrows are the wheel joint axes

7. Building the Simscape Multibody Model

The raw import is a rigid, welded robot. `build_scout_amr_model.m` turns it into a dynamic plant by the following steps, which are executed in the order given and saved as `scout_amr_sim.slx`.

- The Weld Joint between base_footprint and base_link is replaced by a 6-DOF Joint, so the chassis can translate and rotate freely in the world.
- The four wheel revolute joints (j1…j4) get TorqueActuationMode = InputTorque (the controller applies a torque to every wheel) and SenseVelocity = on.
- Transform Sensor blocks measure the world-frame pose and velocity of the chassis and of each wheel (position, velocity, rotation, angular velocity), and Simulink-PS / PS-Simulink converters connect them to the signal domain.
- Four External Force and Torque blocks, one per wheel, apply the tyre forces computed by the tyre-ground model.
- The controller and the tyre model are MATLAB Function blocks that call `scout_amr_controller.m` and `scout_amr_tires.m`. They run at the control period of 5 ms through zero-order-hold blocks.
- The 3D warehouse and the moving obstacles are added as Brick and Cylinder Solid blocks (section 8).
- Logging blocks store the controller, tyre, LiDAR, path and mover signals (To Workspace).

- Solver: `ode23t` (variable step, maximum step 0.01 s), stop time 120 s; the simulation stops itself when the mission is done.

7.1 Tyre-ground contact model

The Scout Mini wheels roll on the floor, so the multibody model needs forces between the wheels and the ground. Simscape Multibody has no built-in friction contact for this case, therefore a contact model was written (`scout_amr_tires.m`) and is evaluated once for every wheel:

- Contact point: below the wheel hub on the axis perpendicular to the axle, at the rolling radius r = 0.085 m. The hub position follows from the wheel frame position plus 0.062 m along the axle.
- Normal force: $F_z = k_c \cdot \delta + c_c \cdot \dot{\delta}$ (never negative) with k_c = 12 000 N/m, c_c = 700 N·s/m per wheel; δ is the penetration of the tyre into the floor.
- Friction: the contact-point velocity gives a sliding velocity; the tangential force is $\mu \cdot F_z$ scaled by a smoothed sign function with smoothing velocity v_0 = 0.03 m/s (smoothed Coulomb friction).
- μ comes from a floor map: μ = 0.9 on dry concrete, 0.15 on the oily loading dock at the start, and 0.30 on two water spills in the aisles.
- Application: the force is resolved in the wheel frame and applied at the wheel-frame origin through the External Force and Torque block, together with the moment of the force about the offset contact point.
- Diagnostics for each wheel: Fz, longitudinal force, lateral force, slip ratio and μ (11 outputs per wheel in the log).

With 30 kg of robot mass the static load per wheel is about 74 N, which compresses the contact by about 6 mm; this agrees with the normal-load trace in Fig. 19 (60–80 N when driving straight).

The model is organised so that its top level stays small. It holds only the World Frame, the Mechanism Configuration, the Solver Configuration and four colour-coded subsystems (Fig. 9); all signals between the subsystems travel through Goto / From tags, so the top level contains no signal wires, only the physical frame connections of the multibody tree.

The subsystem Scout_Mini_Robot holds the imported robot (chassis, wheels, sensor mounts), the sensors, the wheel drives and the nested subsystem Tyre_Ground_Contact_Model with one MATLAB Function block per wheel. The `tc_on` switch is at scout_amr_sim/Mission_and_Slip_Controller/`tc_on`. The blocks of each subsystem are listed in the table below.

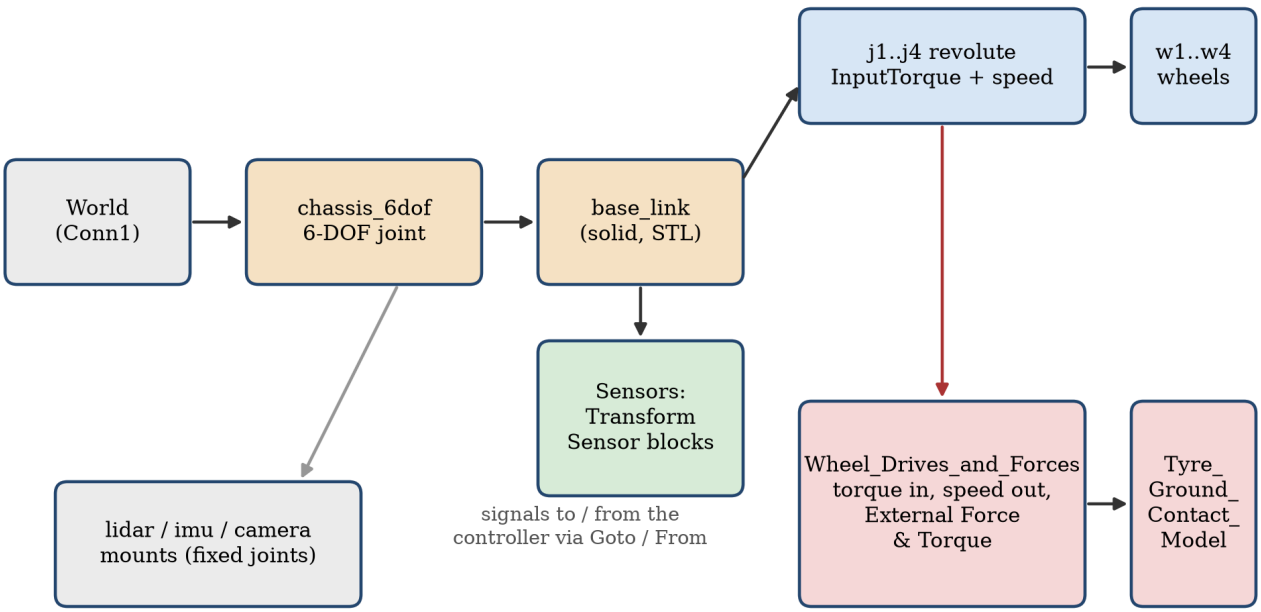


Fig. 8: Inside the subsystem Scout_Mini_Robot (schematic): free 6-DOF chassis, torque-driven wheel joints, sensors, wheel forces from the tyre model

Subsystem	Blocks	Role
Scout_Mini_Robot	base_footprint, chassis_6dof, base_link, w1…w4, j1…j4, lidar / imu / camera mounts, wheel Transform Sensors, tau1…tau4 drives, WheelWrench1…4 (External Force and Torque), Tyre_Ground_Contact_Model	Imported robot with free chassis, driven wheels, sensors and tyre forces
Warehouse	Floor, 3 low-μ patches, 3 pads, 9 racks (+ 27 goods blocks), 4 walls, 2 floor boxes	3D static environment
Moving_Obstacles	Worker_1, Worker_2, Second_AMR, Pallet_Truck with two prismatic joints each, mover demux	Prescribed-motion obstacles
Mission_and_Slip_Controller	`tc_on`, zero-order holds, Controller (MATLAB Function), logging blocks	Autonomy, slip management, logging

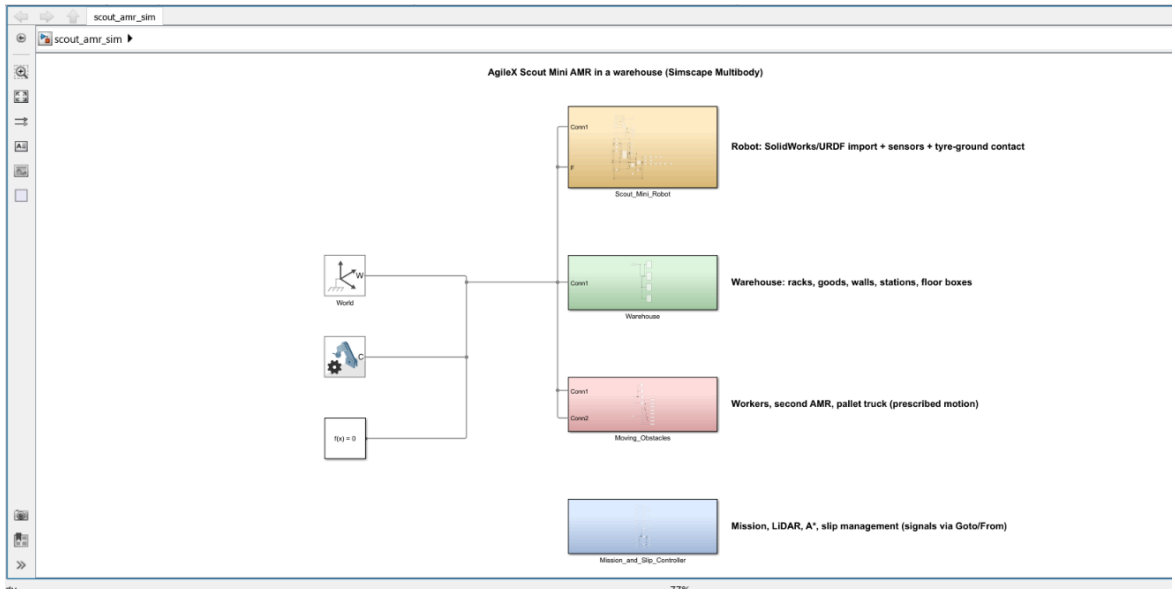


Fig. 9: Top level of scout_amr_sim.slx in Simulink: World, Mechanism Configuration, Solver Configuration and the four subsystems Scout_Mini_Robot, Warehouse, Moving_Obstacles and Mission_and_Slip_Controller

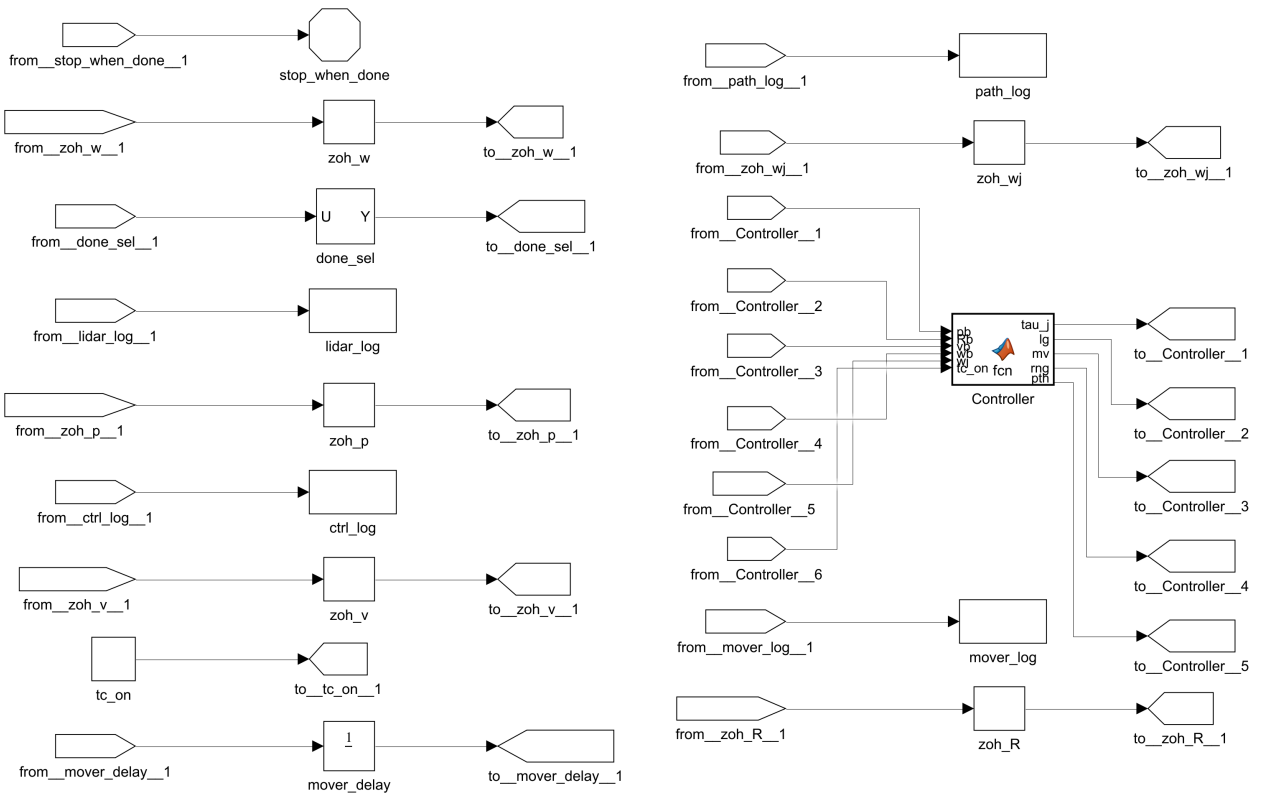


Fig. 10: Subsystem Mission_and_Slip_Controller (zero-order holds, controller function, logging; tc_on selects slip management). The subsystem is shown in two halves: left half first, then right half

Every signal enters the subsystem through a From block and leaves through a Goto block. The zero-order holds (zoh_p, zoh_v, zoh_w, zoh_R, zoh_wj) sample the chassis position, velocity, wheel speeds, rotation and joint speeds at the control period of 5 ms. The Controller block is a MATLAB Function block that calls `scout_amr_controller.m`: its inputs are the sampled states and the constant `tc_on` (1 = slip management on, 0 = off), its outputs are the four wheel torques (tau_j), the 44-value log vector (lg), the mover positions (mv), the LiDAR ranges (rng) and the planned path (pth). The logging blocks (ctrl_log, lidar_log, path_log, mover_log) store these signals for the figures, mover_delay is the Unit Delay that removes the algebraic loop, done_sel selects the mission-done flag, and stop_when_done ends the simulation when the mission is complete.

8. Creating the Warehouse

The warehouse is created twice from the same parameters (`scout_amr_params.m`): as Simscape solids, so that it is part of the 3D multibody scene, and as geometry for the LiDAR, the planner and the collision monitor (`scout_wh_world.m`). The layout is a 31 m × 14.2 m hall with three rack rows, three aisles, two cross-aisles and open docks at both ends. The robot starts at the origin on the dock, facing +x. All dimensions were chosen by the author for the demonstration.

Element	Layout
Hall	x −2.5…28.5 m, y −2.2…12 m, walls 1.2 m high
Rack rows	Row A y 1.5…2.7 m, row B y 5.7…6.9 m, row C y 9.9…11.1 m; each row in three segments at x 3…8.6, 10.2…15.8 and 17.4…23 m (2.4 m high frames with 3 shelves of goods)
Aisles	Aisle 1 y −1.5…1.5 m (service lane), aisle 2 y 2.7…5.7 m, aisle 3 y 6.9…9.9 m; cross-aisles at x 8.6…10.2 and 15.8…17.4 m
Stations	Start pad (0, 0); pickup station (1.2, 4.2) m at the dock end of aisle 2; delivery station (26, 8.4) m at the dock end of aisle 3; loading and unloading time 3 s each
Static obstacles not in the planner's map	Floor box 0.8 × 0.8 m in aisle 2 at (6.5, 3.9) m; floor box in aisle 3 at (13, 8.9) m
Low-friction patches	Oil, μ = 0.15: x −1.5…2.5, y −1.5…1.5 (start dock). Water, μ = 0.30: x 9…11, y 2.7…5.7 (aisle 2) and x 15.5…18, y 6.9…9.9 (aisle 3)
Moving obstacle 1	Worker 1 (cylinder d = 0.6 m) repeatedly crossing aisle 2 at x = 14 m, pausing at the shelves and waiting while the AMR is close

Moving obstacle 2	Worker 2 walking along aisle 3 at y = 7.5 m (x = 15.5…24.5 m), stops when the AMR is within 2.2 m
Moving obstacle 3	Second AMR (0.9 × 0.6 m) shuttling along aisle 1 at y = −0.4 m
Moving obstacle 4	Pallet truck (1.4 × 2.6 m) parked at the east dock; released when the robot passes x = 12 m, it drives into aisle 2 at 1 m/s and stops at x = 20.4 m, blocking the aisle

The moving bodies are Simscape solids on two prismatic joints each (x and y). The positions come from the world model through motion-input joints (filtered, second order), so they move inside the multibody scene. A Unit Delay in the feedback path removes the algebraic loop between the controller and these joints.

8.1 Planning map

The planner works on an occupancy grid with 0.2 m cells (155 × 71 cells). The racks and walls are known in advance (static layer). Boxes, workers, the second AMR and the truck are not in the prior map; they enter the grid only when the LiDAR sees them (dynamic layer, held for 2 s). All occupied cells are inflated by 0.48 m (robot collision radius 0.36 m plus a margin).

8.2 The warehouse in the Multibody Explorer

When scout_amr_sim is run or updated, the Multibody Explorer opens with the 3D scene of the Simscape model. Its model tree shows the four parts of the model (Moving_Obstacles, Scout_Mini_Robot, Warehouse, MechanismConfiguration) and the World frame, and the playback bar runs over the simulated 49.58 s of the mission. Fig. 11 shows the scene at t = 0 in the isometric view with the racks and goods, the blue low-friction patches, the pickup and delivery pads, the floor boxes and the moving obstacles. Fig. 12 is the top view at t = 3.6 s, when the Scout Mini has left the start dock towards the pickup station, and Fig. 13 shows the isometric view at t = 18.7 s, with the robot in aisle 2 and the pallet truck at the east dock. The pictures are screenshots of the Multibody Explorer window.

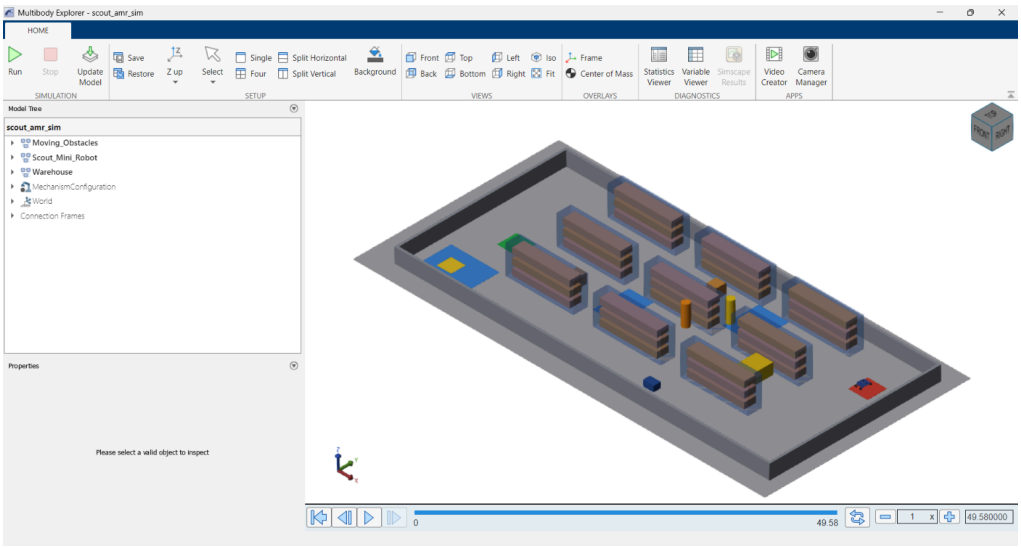


Fig. 11: Multibody Explorer of scout_amr_sim, isometric view at t = 0 (Scout Mini at the start dock)

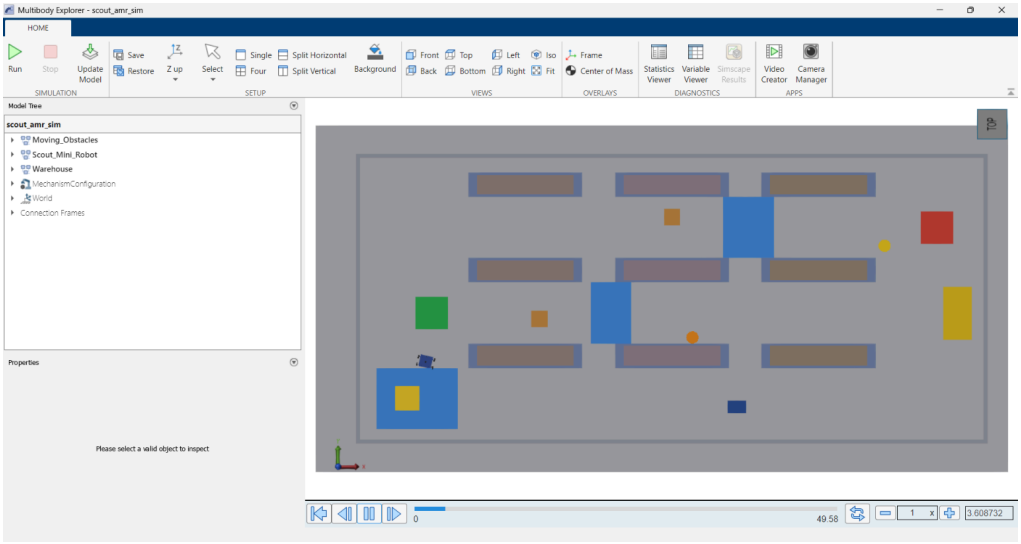


Fig. 12: Multibody Explorer, top view at t = 3.6 s (robot leaving the start dock)

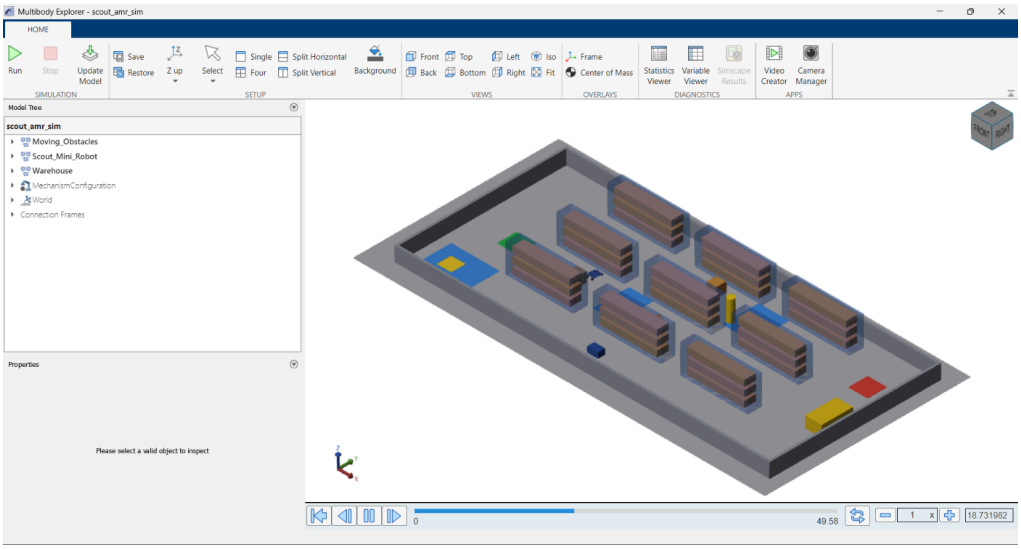


Fig. 13: Multibody Explorer, isometric view at t = 18.7 s (robot in aisle 2)

9. Autonomous Navigation

Autonomous navigation is part of the Simulink model: the MATLAB Function block Controller calls `scout_amr_controller.m` every 5 ms, which calls `scout_wh_nav.m` for the perception, planning and mission logic (Fig. 10). The simulation is started with `run_scout_warehouse_simulation.m`, which also produces all plots (the commands are in section 14).

9.1 One mission from start to delivery

The mission has five states: drive to the pickup station, load for 3 s, drive to the delivery station, unload for 3 s, done. The robot reached the pickup station at 6.1 s, left it at 9.1 s and stopped at the delivery station at 46.6 s. The run ended at 49.6 s. The simulation was repeated with the same settings and gave the same numbers (the world, the workers and the controller are deterministic, there is no sensor noise).

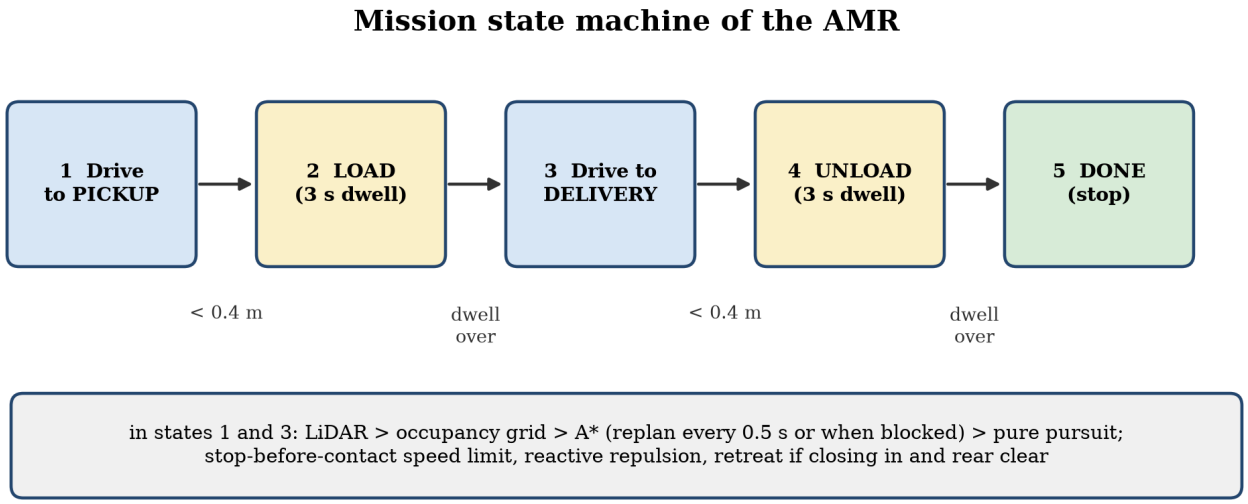


Fig. 14: Mission state machine

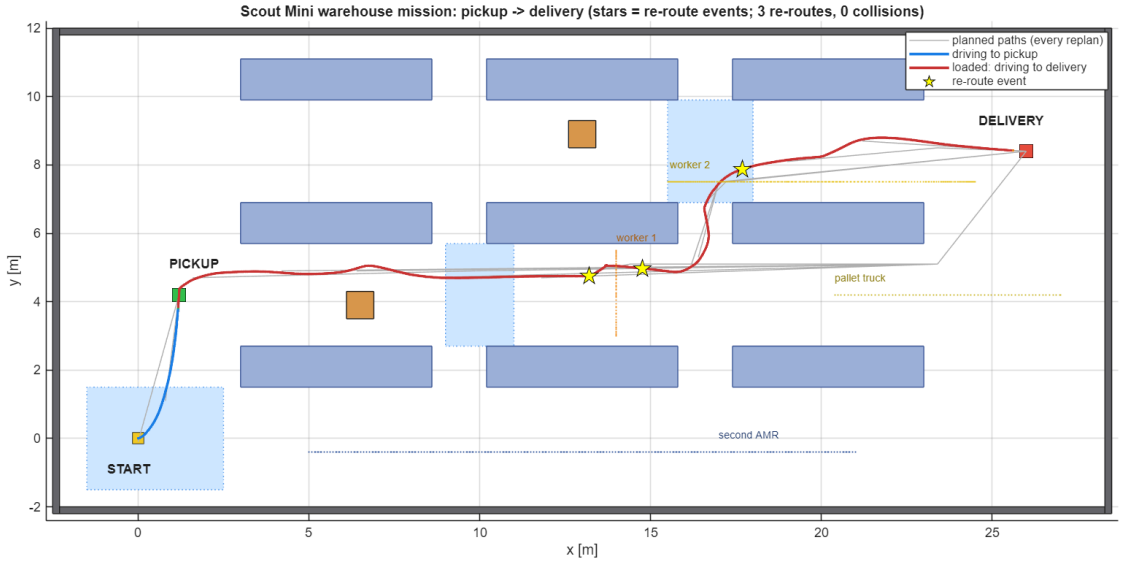


Fig. 15: Warehouse map with the driven trajectory (blue: to pickup, red: to delivery), all planned paths (grey) and the re-route events (stars); dotted lines are the paths of the moving obstacles

The six demonstration scenarios happen in one mission. The table below gives the time of each one in the reference run.

No.	Scenario	Time in the run	What happens
1	Aisle traversal	9–46 s	The robot drives along aisle 2 and then aisle 3 between the racks with about 0.2–0.6 m of clearance
2	Navigating around shelves	≈30–38 s	The robot leaves aisle 2 through the cross-aisle at x ≈ 16.5 m and continues in aisle 3
3	Avoiding a floor box	≈17 s	The LiDAR detects the box in aisle 2; the planner adds it to the map and the path bends around it
4	Avoiding a moving human	≈29 s	Worker 1 crosses aisle 2; the robot slows down and waits or steers around until the lane is clear
5	Pickup to delivery	9.1–46.6 s	Loaded drive from the pickup station to the delivery station
6	Alternative path when blocked	25.1, 30.2, 37.6 s	The pallet truck drives into aisle 2 and stops at x = 20.4 m (at about 27 s), closing the aisle; the planner then routes the robot through the cross-aisle at x ≈ 16.5 m into aisle 3. The three route corrections happen when the current path is blocked by a detected moving obstacle: the first two with worker 1 about 1 m ahead in aisle 2, the third in aisle 3 with worker 2 and the truck within 5 m. The robot backs away when something closes in

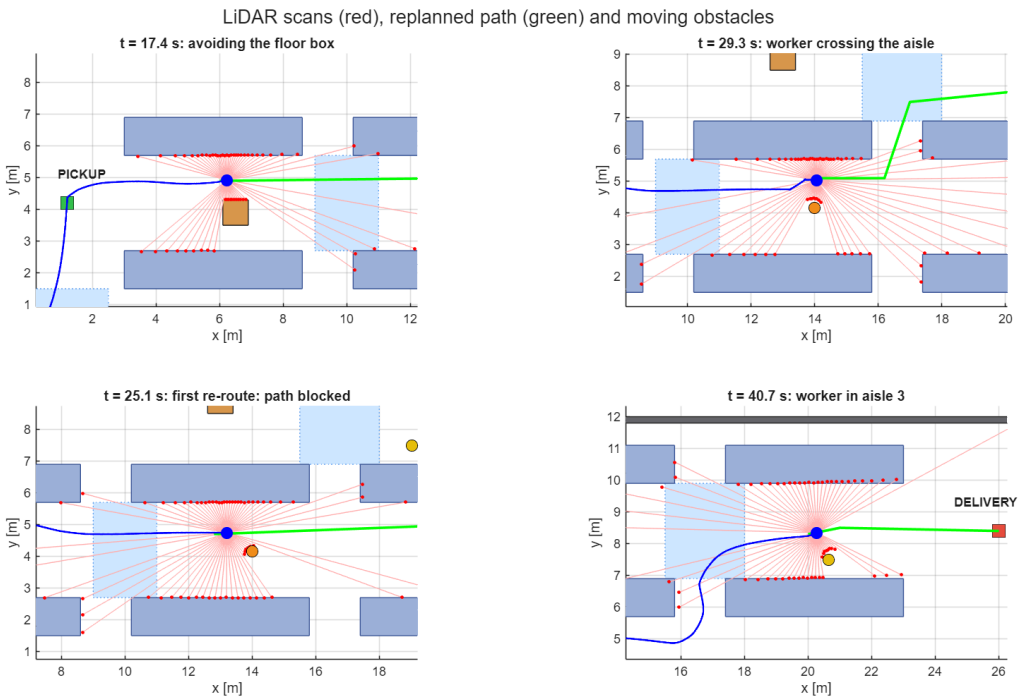


Fig. 16: LiDAR scans (red), re-planned path (green), driven path (blue) and moving obstacles at four moments of the mission

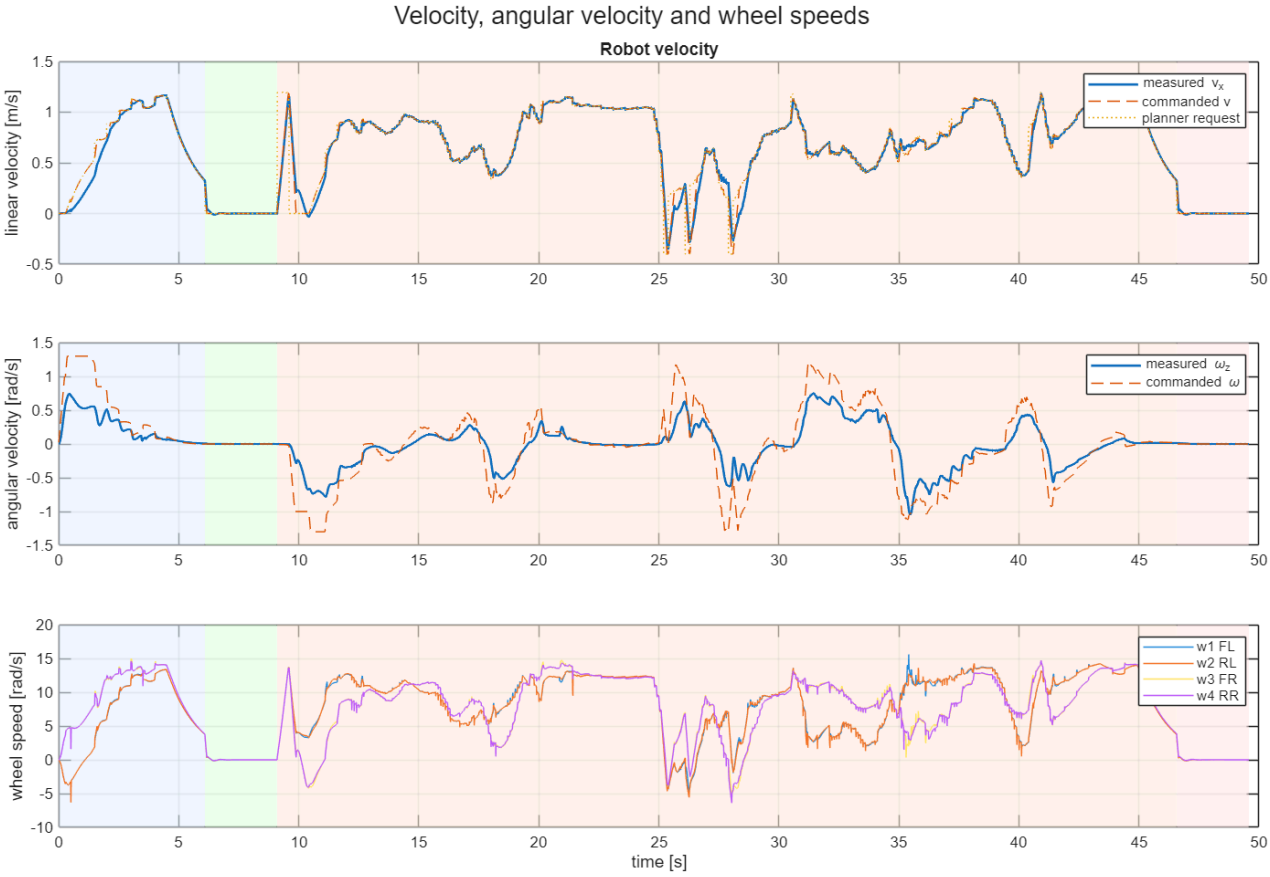


Fig. 17: Linear and angular velocity (commanded and measured) and the four wheel speeds

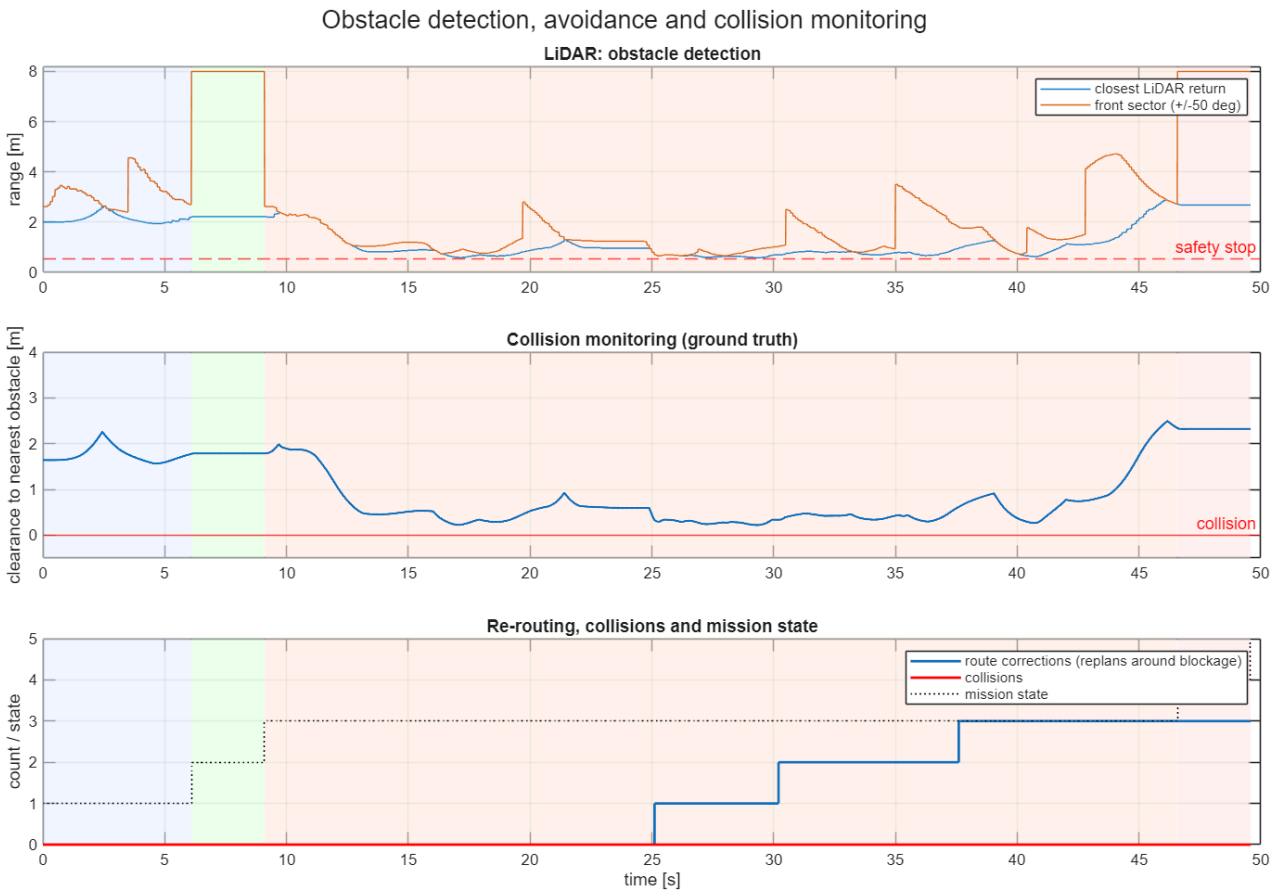


Fig. 18: LiDAR ranges, ground-truth clearance, route corrections and mission state (shaded: drive to pickup, load, drive to delivery, unload)

Quantity	Value
Mission completed	yes
Mission time	49.6 s
Distance driven	32.0 m
Mean / top speed	0.64 m/s / 1.20 m/s
Route corrections (replans around a blockage)	3
Collisions (collision circle r = 0.36 m overlaps an obstacle)	0
Minimum clearance to any obstacle	0.22 m
Closest LiDAR return	0.58 m
Peak wheel slip ratio / RMS slip ratio	0.48 / 0.06
Peak yaw rate	1.04 rad/s
Wheel mechanical work ($\Sigma\lvert\tau\cdot\omega\rvert$)	2670 J
RMS wheel torque / peak wheel torque	1.97 N·m / 6.17 N·m
Wall-clock time of the simulation run	about 80 s for the slip-management-on run and about 25 s for the off run on the author's computer (headless); this depends on the computer

9.2 Presentation outputs and chassis check

`make_example_style_outputs.m` produces a top-down trajectory map and an animation (`amr_warehouse_avoidance.mp4`) from the saved results of the reference run. The same script also checks the chassis: its height settles about 0.7 mm below its starting height within the first second and then stays constant, and the tilt stays within 0.6°, which confirms that the contact model carries the robot without floating or sinking. The clearance, the controller commands and the wheel speeds that the script also plots are shown in Fig. 17 and Fig. 18.

9.3 Limits of these results

- Simulation only; no run on the physical robot. The pose is the true pose of the simulator (no localisation or odometry error).
- One layout and one mission. The simulation is deterministic, so repeating it gives the same numbers; the result shows that the chain works, not that it works on every layout.
- The workers are partly cooperative: they pause when the AMR is close. They are still obstacles that the robot must detect and avoid, but they are gentler than real traffic.
- The LiDAR and the collision check are analytic computations on the warehouse geometry. The robot does not collide physically with the racks or the movers: there is no Simscape contact between the robot and the obstacles, and the movers cannot be pushed.
- The minimum clearance of 0.22 m is measured from the robot's collision circle (r = 0.36 m), which is a simplification of the real chassis outline.

10. Wheel Slip and Traction Management

The floor of the warehouse is not uniform, so the wheels can lose grip. The simulation includes three friction zones (dry concrete, an oily loading dock at the start and water spills in two aisles), and a skid-steer robot also scrubs its wheels sideways in every turn. The controller therefore monitors the slip of every wheel and manages it.

10.1 How slip is detected and managed

The slip ratio of each wheel is calculated from the wheel speed and the ground speed at the wheel:

$$\kappa = (r\cdot\omega - v_k) / \max(|r\cdot\omega|, |v_k|, 0.8\ m/s)$$

Here ω is the wheel speed, r the wheel radius and v_k the ground speed at the wheel's lateral position (chassis speed minus yaw rate times the wheel offset). The 0.8 m/s floor in the denominator avoids division by zero at standstill. The slip is detected and managed as follows:

- Detection: a wheel slips when $|\kappa| > 0.12$; a slip timer records how long this lasts.
- Traction-control window: the wheel-speed target is limited to the speed that matches the ground ± a tolerance (anti-spin on acceleration, anti-lock on braking). The allowed slip is regulated to about 0.08.
- Torque limiting: above the threshold the available torque is reduced linearly, down to 10 % of the 17.5 N·m limit at $\kappa = 0.40$.
- Friction estimation: while slip is detected the friction estimate $\hat{\mu}$ decreases by 0.5 per second (not below 0.10); when there is no slip it recovers at 0.15 per second up to 0.9.
- Friction-aware acceleration: the acceleration demand is limited to $0.85\cdot\hat{\mu}\cdot g$, so the robot accelerates gently on a slippery floor.
- Wheel-speed control: a PI controller (Kp = 0.3, Ki = 8) with an acceleration feed-forward term (30 kg · a · r / 4) drives each wheel to its target; the integrator is clamped to the current torque limit.

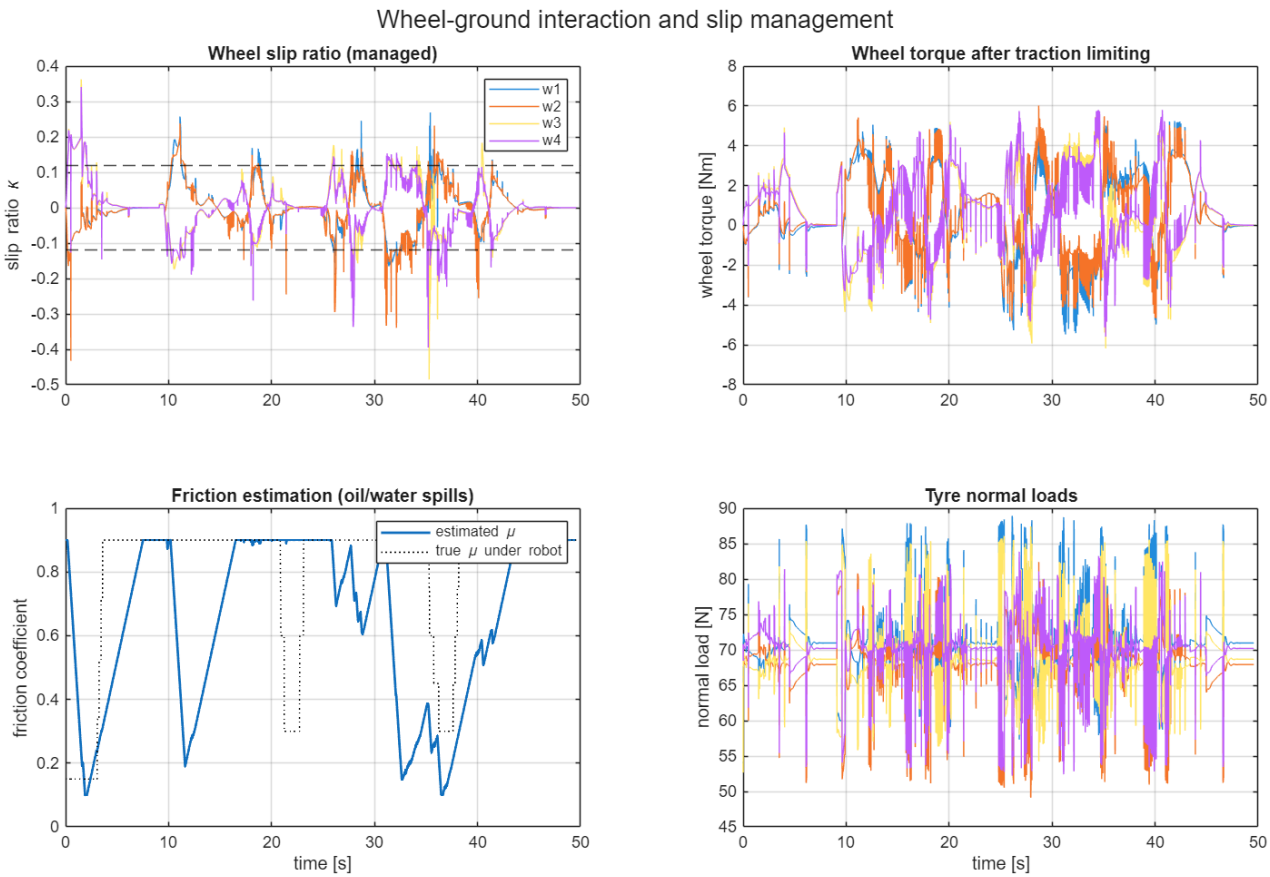


Fig. 19: Wheel slip ratios with the ±0.12 threshold, wheel torques after limiting, friction estimate against the true friction under the robot, and tyre normal loads

The friction estimate falls as low as 0.10 in the low-friction zones and recovers on dry floor, following the true friction with a lag of several seconds (Fig. 19, bottom left). The slip ratio stays mostly within ±0.2 with short spikes during sharp turns, reversing and in the spills. The torque and normal-load traces are noisy because the contact model is stiff and the controller works at 5 ms; this chatter was not tuned away.

10.2 Slip management on and off

The mission was simulated twice with the same world: with the slip management of section 10.1 (`tc_on` = 1) and with a plain wheel-speed controller (`tc_on` = 0), which has no traction window, no torque limiting and no friction-aware acceleration limit.

Quantity	Slip management ON	Slip management OFF
Mission completed	yes	yes
Mission time [s]	49.6	45.5
Distance driven [m]	32.0	31.5
Peak slip ratio (worst wheel)	0.48	0.64
RMS slip ratio (all wheels)	0.064	0.070
Share of time with worst-wheel $\|\kappa\| > 0.2$	2.1 %	5.4 %
Wheel mechanical work [J]	2670	2633
Collisions	0	0
Minimum clearance [m]	0.22	0.19

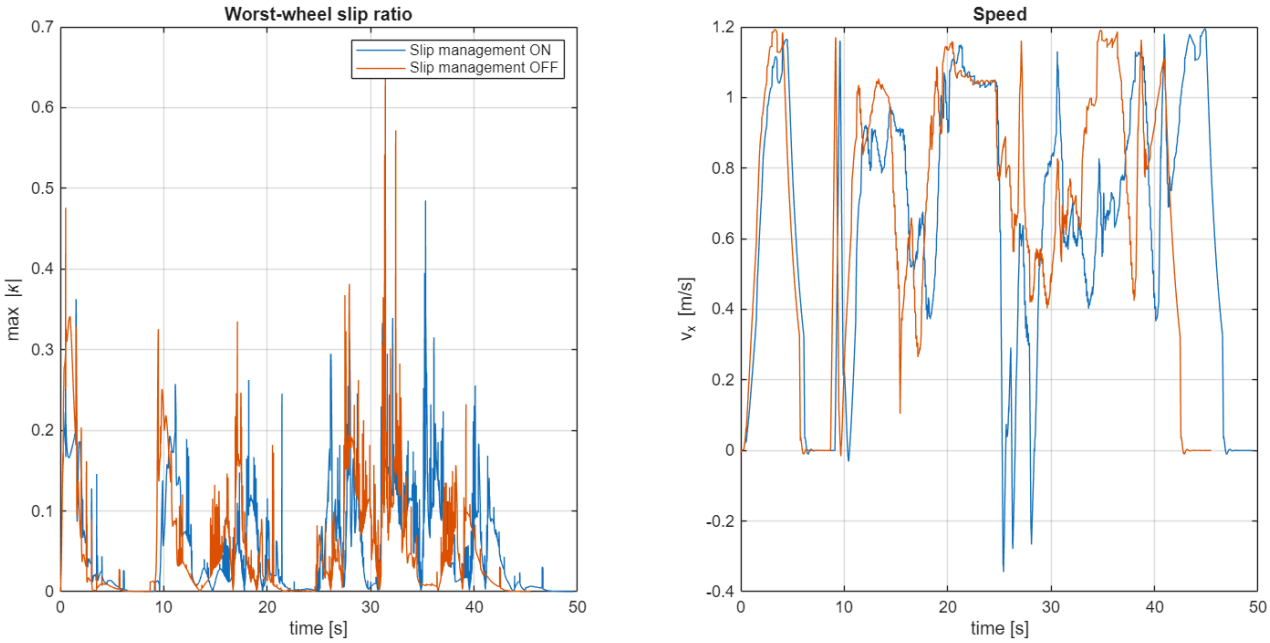


Fig. 20: Worst-wheel slip ratio and forward speed with slip management on (blue) and off (orange)

With slip management the peak slip ratio is lower (0.48 against 0.64) and the worst wheel spends less time above a slip ratio of 0.2 (2.1 % against 5.4 % of the time). The price is time: the friction-aware acceleration limit and the torque limiting make the robot slower, so the mission takes 49.6 s instead of 45.5 s. The energy and the RMS slip ratio are almost the same. In this layout the plain controller also finished the mission without a collision, so the benefit of the slip management is smoother, more controlled wheel-ground behaviour, not the difference between success and failure. A single run per case on one layout does not allow a stronger statement.

11. Algorithms Used

The navigation and control stack is written in MATLAB and runs inside the Simulink model. SLAM, AMCL, Nav2 planners and behaviour trees were NOT used in the Simulink project; their roles are filled by the occupancy grid and A* (global planning), pure pursuit with reactive repulsion (local control) and a simple mission state machine.

Algorithm	File	What it does	Main parameters
Analytic LiDAR	`scout_wh_lidar.m`	Casts rays from the robot against rectangles (racks, walls, boxes, AMR, truck) and circles (workers) and returns the first hit	72 beams, 8 m, 10 Hz, no noise
Occupancy grid with inflation	`scout_wh_nav.m`	Static map plus a dynamic layer built from LiDAR hits; occupied cells are dilated by the robot radius plus a margin	0.2 m cells, 155 × 71, inflation 0.48 m, dynamic hold 2 s
A* global planner	`scout_wh_plan.m`	8-connected A* with octile-distance heuristic and a binary heap; the path is smoothed by line-of-sight shortcuts and limited to 80 points	replan every 0.5 s or when the path is blocked
Pure pursuit	`scout_wh_nav.m`	Steers towards a point on the path ahead of the robot	look-ahead 0.9 m, $v \le 1.2$ m/s
Reactive repulsion and stop-before-contact	`scout_wh_nav.m`	Pushes the robot away from close LiDAR hits and caps the speed at $v = \sqrt{(2 \cdot 0.8 \cdot (\text{range} - 0.55))}$ in front of obstacles	braking 0.8 m/s², stop margin 0.55 m
Retreat behaviour	`scout_wh_nav.m`	If an obstacle closes in and the rear is free, the robot backs away and re-plans	stuck detection, 4 s re-route debounce
Mission state machine	`scout_wh_nav.m`	to pickup → load → to delivery → unload → done	dwell 3 s, goal radius 0.40 m
Skid-steer kinematics	`scout_amr_controller.m`	$\omega_{left} = (v - \tfrac{1}{2} \cdot track \cdot \omega)/r$, $\omega_{right} = (v + \tfrac{1}{2} \cdot track \cdot \omega)/r$; same target for both wheels on a side	track 0.612 m, r = 0.085 m
Wheel-speed PI + feed-forward	`scout_amr_controller.m`	Torque per wheel from speed error and demanded acceleration	Kp 0.3, Ki 8, 17.5 N·m limit, Ts 5 ms
Slip management	`scout_amr_controller.m`	Slip ratio, traction window, torque limiting, friction estimate (section 10)	κ thresholds 0.12 / 0.08 / 0.40

Tyre-ground model	`scout_amr_tires.m`	Spring-damper normal force and smoothed Coulomb friction per wheel	kc 12 000 N/m, cc 700 N·s/m, v0 0.03 m/s
Collision monitor	`scout_wh_nav.m`	Overlap test of the robot circle against all obstacles; logs the clearance	r = 0.36 m

11.1 A* global planner

A* expands the cell with the lowest f(n) = g(n) + h(n), where g is the cost so far and h the octile distance to the goal (the exact cost on an obstacle-free grid with moves of cost 1 and √2), which never overestimates the remaining cost. A move costs 1 for a straight step and √2 for a diagonal step. A binary heap holds the open set. After the search, line-of-sight shortcuts remove unnecessary waypoints, so the path becomes a few straight segments (the grey paths in Fig. 15).

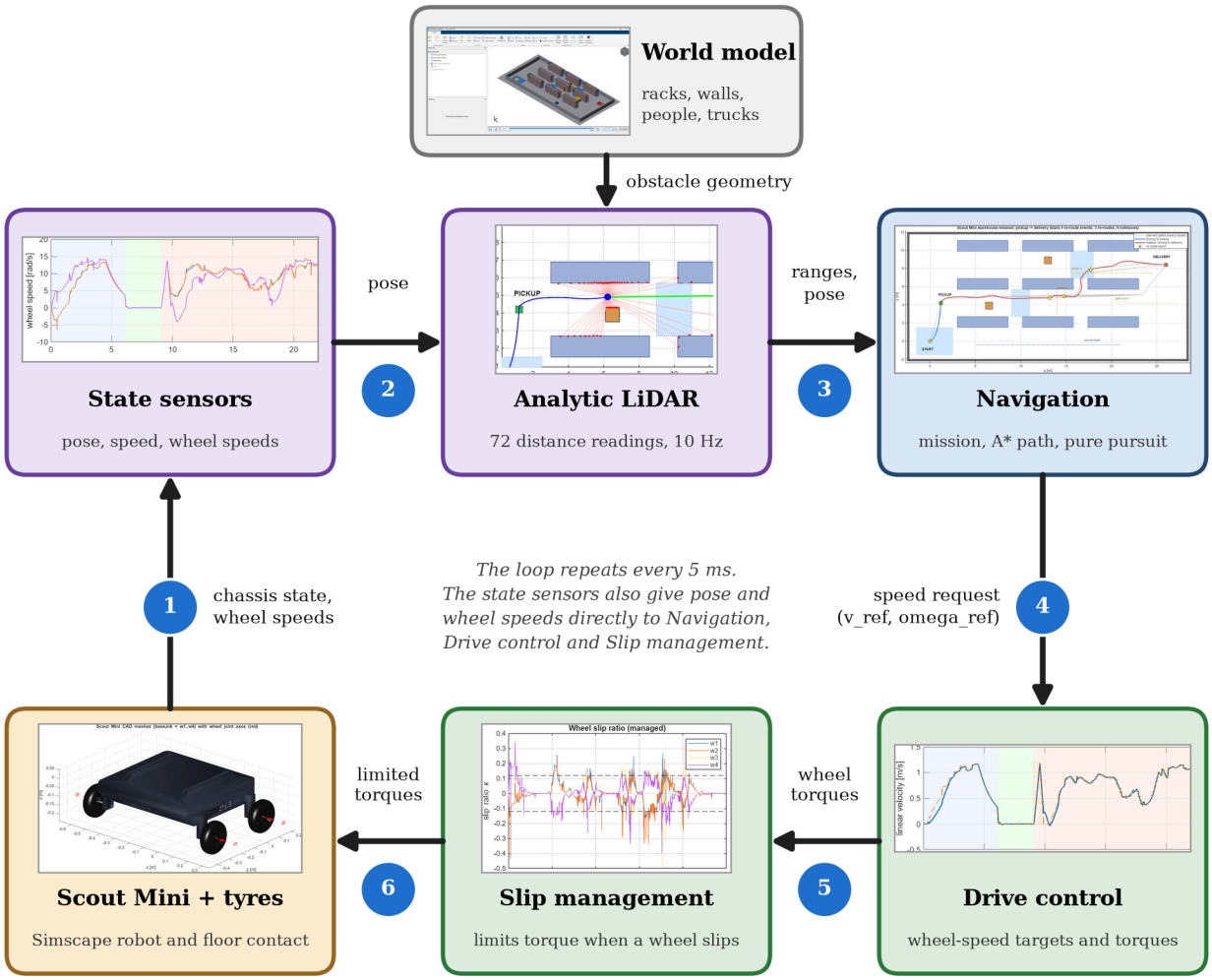
11.2 Dynamic obstacles

Moving obstacles are handled in a reactive way. Every LiDAR scan marks the hit cells in the dynamic layer for 2 s, and the planner re-plans every 0.5 s or at once when the current path crosses an occupied cell. The motion of a moving obstacle is not predicted. If the path is blocked and an obstacle is closing in, the robot stops or backs away before it re-plans, because the stop-before-contact limit never allows a speed from which the robot could not stop in front of the nearest return.

12. System Architecture

Fig. 21 shows how the parts of the simulation are connected. It is one loop that runs clockwise: the robot is measured, the measurements are turned into obstacle readings, the navigation decides where to go, the controller turns that into wheel torques, and the torques move the robot, which is measured again. Follow the blue numbers 1 to 6; after 6 the loop starts again at 1. One round takes 5 ms.

- Sensors and LiDAR (steps 1 to 3): the state sensors give pose, speed and the four wheel speeds; the analytic LiDAR uses the pose and the obstacle geometry of the world model (`scout_wh_world.m`) to produce 72 ranges at 10 Hz. The LiDAR does not scan the Simscape solids; the same world model moves the 3D warehouse movers.
- Navigation (step 3 and 4): mission state machine, A* planner and pure pursuit give a speed and yaw-rate request.
- Control (steps 4 to 6): the drive control converts the request into wheel-speed targets and torques, and the slip management limits them (at most 17.5 N·m per wheel).
- Robot (step 6 and 1): the Scout Mini and the tyre-ground contact model (Simscape Multibody) react to the torques. The state sensors also give pose and wheel speeds directly to the navigation, the drive control and the slip management.
- Not in the loop: the collision monitor only logs the ground-truth clearance. There is no localisation block, the pose is the true simulator pose.



Not in the loop: the collision monitor only logs the clearance to obstacles and is never used by the controller.

Fig. 21: Closed-loop architecture of the Simulink / Simscape Multibody simulation; the numbered arrows are the steps of one control cycle (table below); the pictures in the blocks are taken from the results of this project

No.	Step	What happens
1	Robot to sensors	The Transform Sensors and the wheel joints give the chassis pose and speed and the four wheel speeds (every 5 ms)
2	Sensors to LiDAR	The pose, together with the obstacle geometry of the world model, goes to the analytic LiDAR, which scans it (72 ranges, 10 Hz)
3	LiDAR to navigation	The ranges and the pose go to the navigation: the occupancy grid is updated, the mission state machine sets the goal, A* plans the path (every 0.5 s or when blocked) and pure pursuit follows it with LiDAR repulsion and the stop-before-contact limit
4	Navigation to drive control	The speed and yaw-rate request (v_ref, omega_ref) goes to the drive control, which converts it into wheel-speed targets with the skid-steer kinematics; the wheel-speed PI and the acceleration feed-forward give the torques
5	Drive control to slip management	The slip management compares the wheel speeds with the ground speed, limits the targets and the torque when a wheel slips, and lowers the acceleration limit on a slippery floor
6	Slip management to robot	The four limited wheel torques (at most 17.5 N·m) drive the wheel joints of the Scout Mini; the tyre-ground model returns the forces F and M, the chassis moves and the new state is sampled in the next 5 ms step (back to step 1)

12.1 One control step

Inside the simulation the controller block runs every 5 ms, as shown in Fig. 22, while the multibody solver (`ode23t`) integrates the plant between the steps.

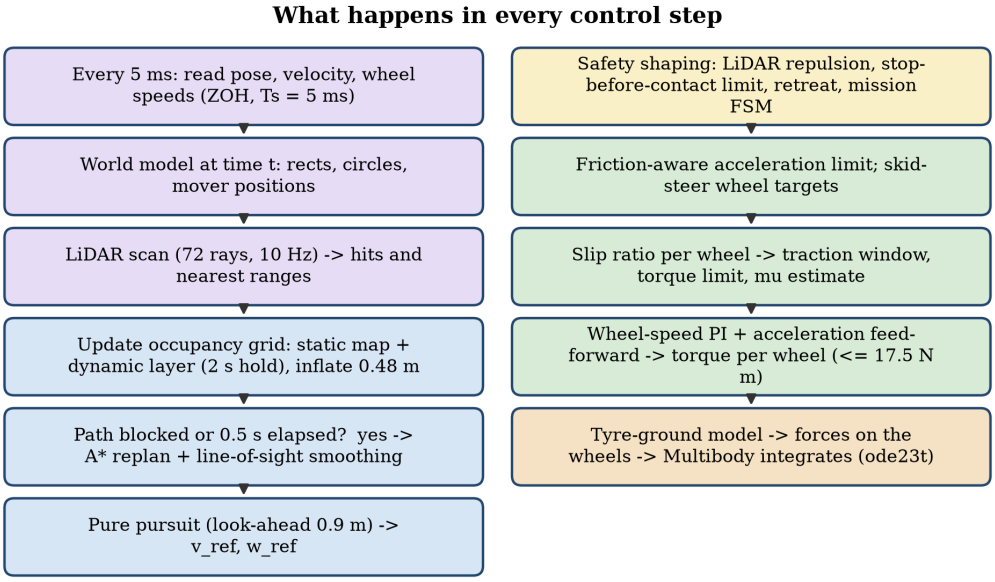


Fig. 22: Flow of one control step

- Every 5 ms: pose, velocity and wheel speeds are sampled; the world is evaluated at the current time (the truck is released when the robot passes x = 12 m); the LiDAR is scanned at 10 Hz; the occupancy grid is updated.
- Every 0.5 s (or at once when the path is blocked): A* is run again from the current pose to the current goal.
- Control: pure pursuit gives the speed and yaw-rate references; they are shaped by the repulsion, the stop-before-contact limit and the friction-aware acceleration limit, converted into wheel targets, and the wheel-speed PI with slip management gives the four wheel torques.
- Physics: the tyre-ground model computes the forces, Simscape Multibody moves the robot, and the movers follow their scripted motion. When the mission state is done the simulation stops itself.

12.2 How the MATLAB files are linked

The files form a small call hierarchy. `scout_amr_params.m` is the common base: every other file reads its constants from it, so a number changed there changes the whole simulation. The Simulink model `scout_amr_sim.slx` is the only place where the files meet the physics: its MATLAB Function blocks call the controller and the tyre model. The three scripts at the bottom of the table build the model, run it and plot the saved results.

```
One run:  run_scout_warehouse_simulation
  |-- scout_amr_params ............... constants P (used by every file below)
  |-- build_scout_amr_model .......... only if scout_amr_sim.slx is missing
  |      |-- smimport(scout_mini.urdf + meshes/) -> scout_import_raw.slx
  |      '-- writes scout_amr_sim.slx with MATLAB Function blocks that call
  |             scout_amr_controller  and  scout_amr_tires
  |-- sim('scout_amr_sim')  (tc_on = 1, then 0 for the comparison)
  |      every 5 ms:
  |      Controller block -> scout_amr_controller
  |            '-- scout_wh_nav
  |                  |-- scout_wh_world   obstacle geometry and mover positions at time t
  |                  |-- scout_wh_lidar   72 ranges (10 Hz)
  |                  '-- scout_wh_plan    A* path (every 0.5 s or when blocked)
  |           returns 4 wheel torques, log vector, mover positions, LiDAR ranges, path
  |      Tire blocks (one per wheel) -> scout_amr_tires -> forces on the wheel
  '-- converts the logs, saves results/scout_warehouse_results.mat, draws Fig. 15 to 20

Afterwards:  make_example_style_outputs reads scout_warehouse_results.mat
             -> trajectory_map.png, signals.png, amr_warehouse_avoidance.mp4 / .gif
```

File	Uses / calls	Called by	Input -> output
scout_amr_params.m	nothing	all other files (make_example_style_outputs via the saved P)	no input -> struct P (robot, tyre, floor patches, warehouse layout, LiDAR, grid, controller gains)
build_scout_amr_model.m	scout_amr_params, smimport on scout_mini.urdf and meshes/ (cached as scout_import_raw.slx)	run_scout_warehouse_simulation (when the .slx is missing) or the user	URDF + parameters -> scout_amr_sim.slx
scout_amr_sim.slx	MATLAB Function blocks: Controller -> scout_amr_controller; Tire1… Tire4 -> scout_amr_tires	run_scout_warehouse_simulation or the user	wheel torques and forces in -> logs ctrl_log, tire_log, lidar_log, path_log, mover_log out
scout_amr_controller.m	scout_amr_params, scout_wh_nav	Controller block, every 5 ms	chassis pose, rotation, velocities, wheel joint speeds, tc_on -> 4 wheel torques, 44-value log vector, 8 mover coordinates, 72 ranges, path
scout_wh_nav.m	scout_amr_params, scout_wh_world, scout_wh_lidar, scout_wh_plan	scout_amr_controller	footprint pose (x, y, yaw) and dt -> speed and yaw-rate request, mover positions, LiDAR ranges, path, info vector (state, route corrections, clearance, collisions)
scout_wh_world.m	scout_amr_params	scout_wh_nav	time t, truck release time, robot position -> rectangles, circles, mover positions
scout_wh_lidar.m	nothing (geometry is passed in)	scout_wh_nav	pose + rectangles + circles -> 72 ranges
scout_wh_plan.m	nothing (grid settings are passed in)	scout_wh_nav	start, goal, blocked grid -> path of up to 80 waypoints
scout_amr_tires.m	scout_amr_params	Tire blocks, one per wheel, every 5 ms	wheel position, velocity, rotation, angular velocity -> force and torque on the wheel, Fz, slip ratio, μ
run_scout_warehouse_simulation.m	scout_amr_params, build_scout_amr_model (if needed), the model scout_amr_sim, its plotting sub-functions	the user	options (3D view, slip comparison) -> results/scout_warehouse_results.mat and Fig. 15 to 20
make_example_style_outputs.m	reads scout_warehouse_results.mat (R and P)	the user	saved run -> trajectory_map.png, signals.png, MP4 / GIF
live_graphs.m	scout_amr_params, build_scout_amr_model (if needed); reads the Controller block outputs of the running model	the user	running simulation -> live figure (map, speed, slip, clearance)

In short: parameters feed everything, the model calls the controller and the tyre model, the controller calls the navigation, the navigation calls the world, LiDAR and planner, and the run script ties model, parameters and plots together. Nothing calls the run script, and nothing outside the model calls the controller or the tyre model, because they are written as MATLAB Function blocks of the Simulink model.

13. Additional MATLAB Visual Simulation (ISO 3691-4 Style)

Before the Simulink model, a MATLAB-script simulation of the same robot was written: simulate_industry_amr_warehouse.m (with plan_warehouse_roadmap.m for the route and teleop_industry_amr_warehouse.m for manual driving). It draws the robot with the same STL meshes in a 3D warehouse and shows how an industrial AMR behaves with safety fields. It is a kinematic and sensor-level model: the robot motion is integrated by the script, not by a multibody solver, and wheel slip is a drift model, not a contact simulation. Wheel-spin direction in the animation was corrected during this work.

Item	Configuration
Warehouse	26 × 16 m, 5 m ceiling; six three-tier racks in two rows (7 × 2 m, 3.6 m high); four static pallets; two extra hazards (crate in the west corridor, pallet in the central cross-aisle); outer walls
Loading dock	Ramp 3.5 × 3 m rising to a dock at 0.45 m; ramp friction 0.78, floor friction 0.68; depot (1.8, 2.0) m, dock (24, 13.5) m, two charging pads
Robot	30 kg + 12 kg payload, chassis 0.62 × 0.55 m, wheel radius 0.08 m, track 0.612 m, $v \le 0.8$ m/s, $\omega \le 1.4$ rad/s, acceleration 0.85 m/s², braking 1.8 m/s², hub-motor torque 17.5 N·m
LiDAR	64 beams, 8 m, $\sigma = 0.015$ m, 1 % dropouts, mounted 0.28 m above the chassis base

IMU and odometry	Gyro bias drift 0.0025 rad/s, noise σ = 0.012; wheel odometry with 2 % systematic drift (shows why LiDAR-based localisation is needed)
Safety fields (ISO 3691-4 style)	Warning 1.80 m (speed derating), braking 1.05 m (path bypass), protective stop 0.65 m (emergency halt)
Planner	`plan_warehouse_roadmap.m`: waypoint roadmap around known and extra hazards; re-routing counter, cool-down time

The script was run unchanged and its window was captured while it ran. The screenshot in Fig. 23 shows the top (plan) camera early in the mission; its right-hand side is the live telemetry of the script (occupancy grid, kinematics, safety clearance). In this run the robot reached the elevated dock after 44.7 s over 32.3 m with the path shown in Fig. 25.

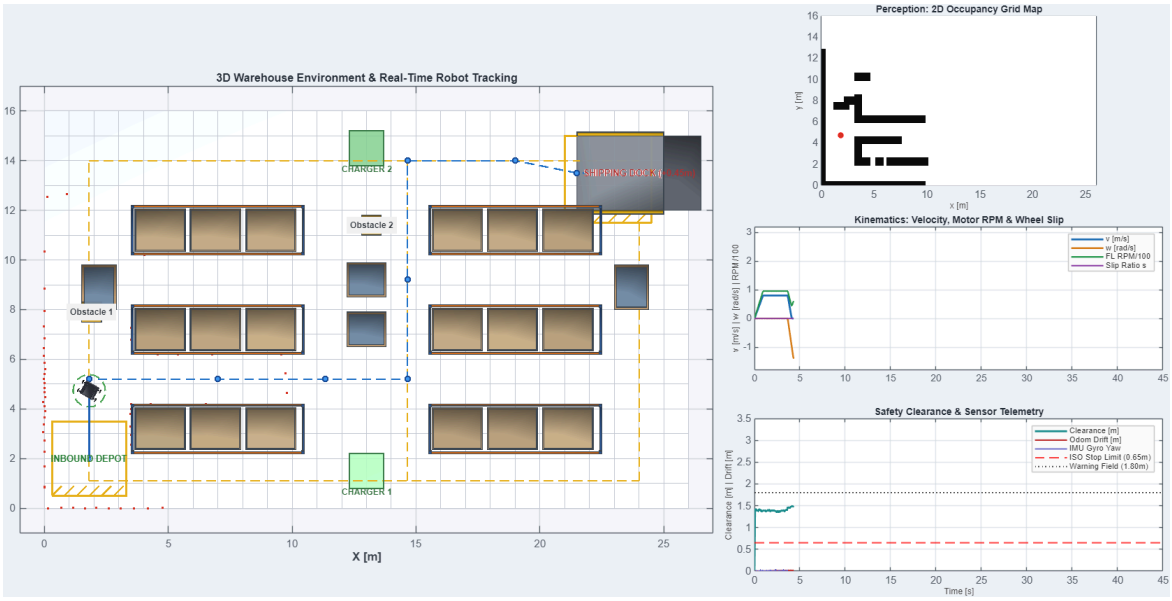


Fig. 23: Top (plan) camera: robot, waypoint roadmap, racks, pallets, charging pads and the shipping dock

The telemetry of the run was extracted from the plots of the script and redrawn as separate graphs. Fig. 24 shows the linear speed (limit 0.8 m/s), the yaw rate (limit 1.4 rad/s) and the motor speed with the wheel slip ratio. The robot stops and pivots at the three corners of the route; the slip ratio stays at zero because the script has a drift model, not a tyre-ground contact simulation (this is the difference to the Simscape model of sections 7 to 10).

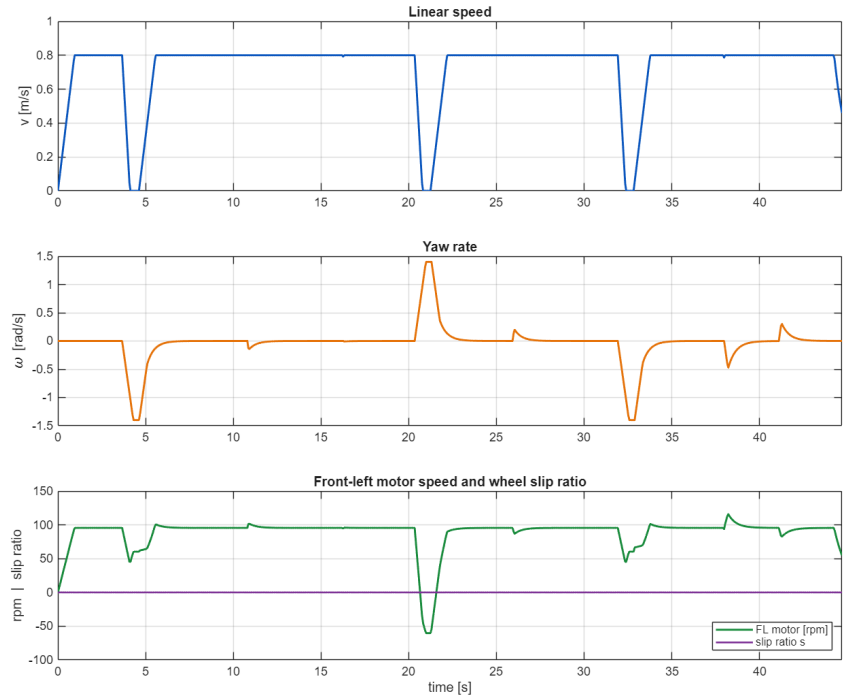


Fig. 24: Linear speed, yaw rate, front-left motor speed and slip ratio of simulate_industry_amr_warehouse.m

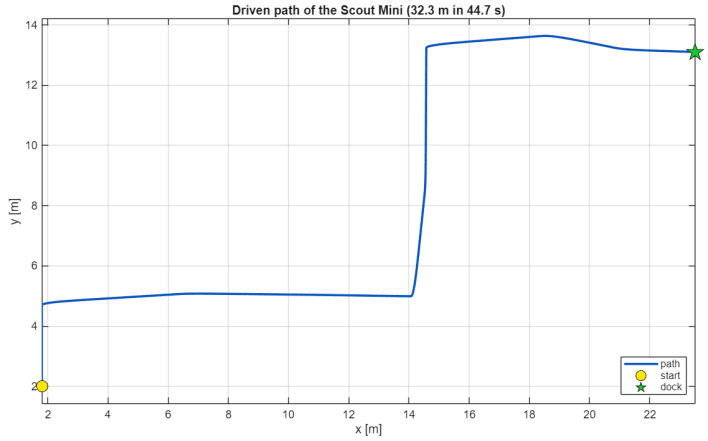


Fig. 25: Path driven by the script from the inbound depot to the shipping dock

Fig. 26 shows the clearance measured by the synthetic LiDAR against the three ISO 3691-4 style fields of the script (warning 1.80 m, braking 1.05 m, protective stop 0.65 m). The clearance falls below the braking field while the robot passes between the racks (6 s to 17 s and 22 s to 32 s) and reaches a minimum of 0.22 m at about 23 s; the script reports no collision. The clearance is the LiDAR range to the nearest hit, so it is not a body-to-obstacle distance and the values below 0.65 m show that the protective-stop field is not a hard limit in this script. The three large yaw-rate peaks of Fig. 24 and Fig. 26 are the turns at the route corners. The odometry drift grows to 0.42 m because of the 2 % wheel-slip drift that was built in, which is why a real robot needs LiDAR-based localisation.

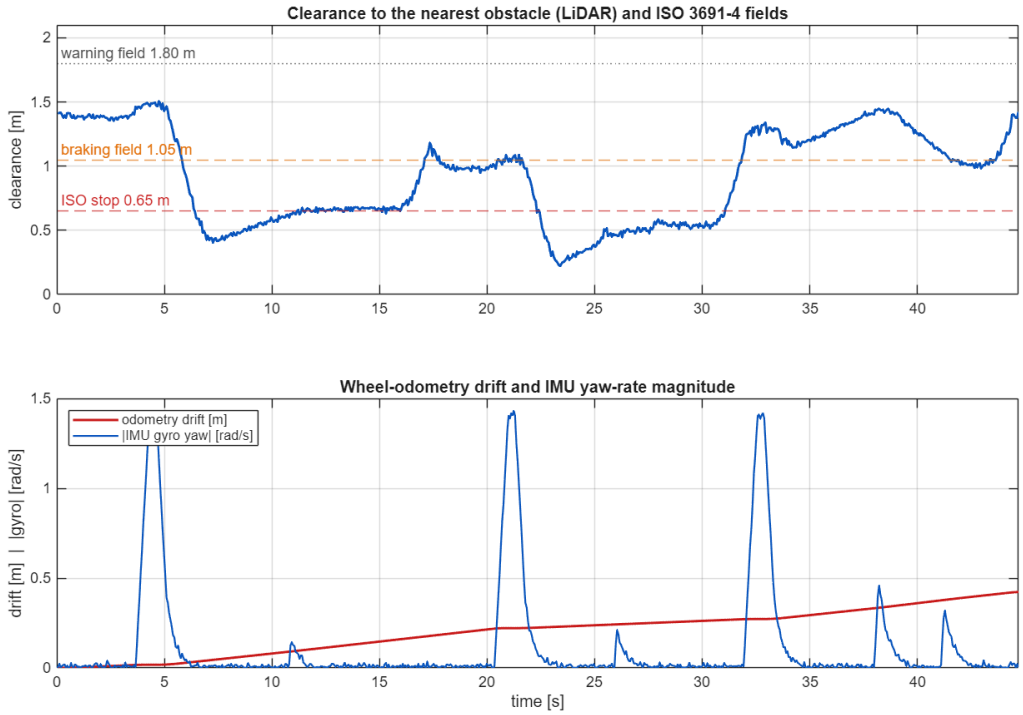


Fig. 26: LiDAR clearance with the ISO 3691-4 fields, odometry drift and IMU yaw-rate magnitude

14. How to Set It Up

14.1 Requirements

Item	Used for
MATLAB R2025b (Windows 11 was used)	All scripts and the Simulink model
Simulink	The model `scout_amr_sim.slx` and the MATLAB Function blocks
Simscape and Simscape Multibody	Multibody blocks, `smimport`, Multibody Explorer (libraries sm_lib and nesl_utility)
Robotics System Toolbox	`importrobot`, `rigidBodyTree` and show (section 6)
SolidWorks and the SolidWorks-to-URDF exporter (only to change the CAD model)	Sections 4 and 5; not needed to run the simulation
Git (optional)	To download the project from GitHub

14.2 Getting the files

The project is in the GitHub repository Saibts/AMR-project. Download it with Git or as a ZIP file and keep the folder structure, because the script simulation in the root folder loads the STL meshes from the meshes folder next to it.

```
git clone https://github.com/Saibts/AMR-project.git

AMR-project/
  simulate_industry_amr_warehouse.m   script simulation (section 13)
  plan_warehouse_roadmap.m            route planner of the script simulation
  meshes/                             STL meshes for the script simulation
  simulink_amr/                       the Simscape Multibody project (sections 5 to 12)
    scout_mini.urdf, meshes/          URDF and STL meshes of the robot
    scout_import_raw.slx              cached smimport result
    scout_amr_sim.slx                 the simulation model
    scout_amr_params.m, scout_amr_controller.m, scout_amr_tires.m, scout_wh_*.m
                                      parameters, controller, tyre model, world, LiDAR,
planner
    build_scout_amr_model.m           rebuilds the model from the URDF
    run_scout_warehouse_simulation.m  runs the mission and draws the graphs
    make_example_style_outputs.m      trajectory map, signals figure, MP4 / GIF
    results/                          figures and scout_warehouse_results.mat
```

14.3 Step-by-step run

Every step below is typed in the MATLAB Command Window. The steps follow the order of this report and name the figures that you should see.

Step	Command	What you should see
1 Open the project folder	cd '...\AMR-project\simulink_amr' addpath(pwd); addpath('..');	No output; the root folder is added so that the script simulation is found
2 Check the URDF import (section 6)	rbt = `importrobot('scout_mini.urdf')`; show(rbt, homeConfiguration(rbt), 'Visuals', 'on');	The robot of Fig. 6 with the base link and four wheels
3 Build the model (sections 6 and 7)	build_scout_amr_model	Uses `scout_import_raw.slx` if it exists; otherwise `smimport` runs first (several minutes) and the edits of section 7 are applied; `scout_amr_sim.slx` is saved. The repository already contains both files, so this step is only needed after changes
4 Open the model (Fig. 9)	scout_amr_params; open_system('scout_amr_sim')	The top level with the four subsystems; open_system('scout_amr_sim/Mission_and_Slip_Controller') shows Fig. 10
5 Run the mission with the 3D view (section 8.2)	`run_scout_warehouse_simulation`	The Multibody Explorer opens (Fig. 11 to Fig. 13) and the robot drives the mission (about 1.5 minutes of wall time); then the six result figures are drawn and the summary of section 9 is printed
6 Run without the 3D window	`run_scout_warehouse_simulation(false)`	The same results, headless, about 1.5 minutes in total
7 Compare slip management on and off (section 10)	`run_scout_warehouse_simulation(false, true)`	Both runs (about 2 minutes) and the comparison figure (Fig. 20); both runs are saved in results/`scout_warehouse_results.mat`
8 Redraw the graphs from the saved results	`run_scout_warehouse_simulation('replot')`	Fig. 15 to Fig. 19 in a few seconds, without simulating
9 Presentation outputs (section 9.2)	make_example_style_outputs	results/trajectory_map.png, results/signals.png and the animation `amr_warehouse_avoidance.mp4` / .gif (several minutes); make_example_style_outputs(false) skips the animation
10 Script simulation (section 13)	cd('..'); simulate_industry_amr_warehouse	The interactive window of Fig. 23 with camera buttons, Replan Route and Emergency Stop; the mission ends after about 45 s
11 Live graphs while the simulation runs (section 14.8)	live_graphs	The Multibody Explorer and one figure that updates during the mission (Fig. 27); live_graphs(false) without the 3D window

14.4 Changing the model

The parameters of the Simulink model are all in `scout_amr_params.m`. The table lists the ones that are useful to change; after a change run the mission again (step 5 or 6). The slip-management switch is the constant `tc_on` in the subsystem Mission_and_Slip_Controller (1 = on, 0 = off).

Parameter	Value	Effect
P.kc, P.cc, P.v0	12 000 N/m, 700 N·s/m, 0.03 m/s	Tyre stiffness, damping and friction smoothing (section 7.1)
`P.mu0` and P.patches	0.9; oil 0.15, water 0.30	Floor friction and the slippery zones (section 8)
P.vmax, P.amax, P.tmax	1.2 m/s, 2.5 m/s², 17.5 N·m	Speed, acceleration demand and motor torque limit
P.Kp, P.Ki	0.3, 8.0	Wheel-speed PI controller
P.k_th, P.k_tgt, P.k_max	0.12, 0.08, 0.40	Slip detection threshold, target slip and slip at minimum torque (section 10.1)
P.lidar.n, P.lidar.rmax	72 beams, 8 m	LiDAR resolution and range
P.grid.inflate, P.grid.replanT	0.48 m, 0.5 s	Obstacle inflation and re-planning interval
P.pickup, P.deliver, P.dwell	(1.2, 4.2) m, (26, 8.4) m, 3 s	Station positions and loading time
P.racks, P.boxes, `P.mover.*`	see section 8	Warehouse layout and the moving obstacles; the warehouse blocks of the model are rebuilt with build_scout_amr_model

14.5 From the SolidWorks model to the URDF

Only needed when the CAD model is changed. The SolidWorks files are in the repository Saibts/AgileX-scout-mini-sw (`Assem2.SLDASM` and the parts). Create the coordinate systems and axes of the four wheel joints in the assembly,

export the assembly with the SolidWorks-to-URDF exporter (this gives the URDF and the STL meshes), copy the URDF to simulink_amr/`scout_mini.urdf`, replace the ROS path `package://assem2_robot/meshes/` by `meshes/` (section 5), copy the STL files to simulink_amr/meshes and run build_scout_amr_model. Delete `scout_import_raw.slx` first so that `smimport` reads the new URDF.

14.6 If something does not work

Problem	Remedy
The Multibody Explorer does not open	set_param('scout_amr_sim', 'SimMechanicsOpenEditorOnUpdate', 'on') and then set_param('scout_amr_sim', 'SimulationCommand', 'update'); look in the taskbar, the window can be behind other windows
Undefined variable or parameter in the model	Run scout_amr_params first, then simulate again
An old copy of the model is open or the model looks different	close_system('scout_amr_sim', 0); bdclose all; open_system('scout_amr_sim')
The figures show only one run or the OFF run is missing	Run `run_scout_warehouse_simulation(false, true)`; a run without the second argument overwrites the saved comparison
simulate_industry_amr_warehouse cannot find plan_warehouse_roadmap or the meshes	Run it from the root folder (cd('..')) with the meshes folder next to it, and addpath('..')
`smimport` is very slow	This is normal (several minutes); the repository includes `scout_import_raw.slx` so that it is not needed again

14.7 Code layout

File	Purpose
`scout_mini.urdf`, `meshes/`	URDF of the robot and the STL meshes (base_link, w1…w4)
`build_scout_amr_model.m`	Imports the URDF (`smimport`) and builds `scout_amr_sim.slx` step by step
`scout_amr_sim.slx`	The Simulink / Simscape Multibody simulation model
`scout_amr_params.m`	All parameters: robot, tyre, friction patches, warehouse layout, LiDAR, grid, controller
`scout_amr_controller.m`	Mission navigation call, skid-steer control, slip management, logging vector
`scout_amr_tires.m`	Tyre-ground contact model (per wheel)
`scout_wh_world.m`	Warehouse geometry and moving obstacles at time t
`scout_wh_lidar.m`, `scout_wh_plan.m`, `scout_wh_nav.m`	LiDAR, A* planner, perception / planning / mission state machine
`run_scout_warehouse_simulation.m`	Runs the simulation, draws all figures, prints the summary
`make_example_style_outputs.m`	Trajectory map, signal figure and animation of the Simscape run
`live_graphs.m`	Runs the mission and updates the map, speed, slip and clearance graphs while the simulation runs (section 14.8)
results/	Figures 1–6, trajectory_map, signals, MP4 / GIF and the saved results (`scout_warehouse_results.mat`)
`../simulate_industry_amr_warehouse.m`, `../plan_warehouse_roadmap.m`	MATLAB-script visual simulation (section 13)

14.8 Live graphs while the simulation runs

`run_scout_warehouse_simulation` draws its figures after the mission has ended, because the model logs its signals to the workspace. For a demonstration in which the graphs move together with the robot, the script `live_graphs.m` runs the same mission and updates one figure about every 0.1 s of wall time. It reads the output ports of the Controller block (the 44-value log vector, the mover positions, the LiDAR ranges and the planned path) while the simulation is running, so the model itself is not changed. Fig. 27 is a screenshot taken at t = 10.3 s, when the robot has loaded at the pickup station and is starting towards the delivery station.

```
>> cd 'D:\academic_shit\SEM5-III\AMR ROS\simulink_amr'
>> live_graphs            % Multibody Explorer (3D) and the live figure, slip management on
>> live_graphs(false)     % live figure only, without the 3D window
```

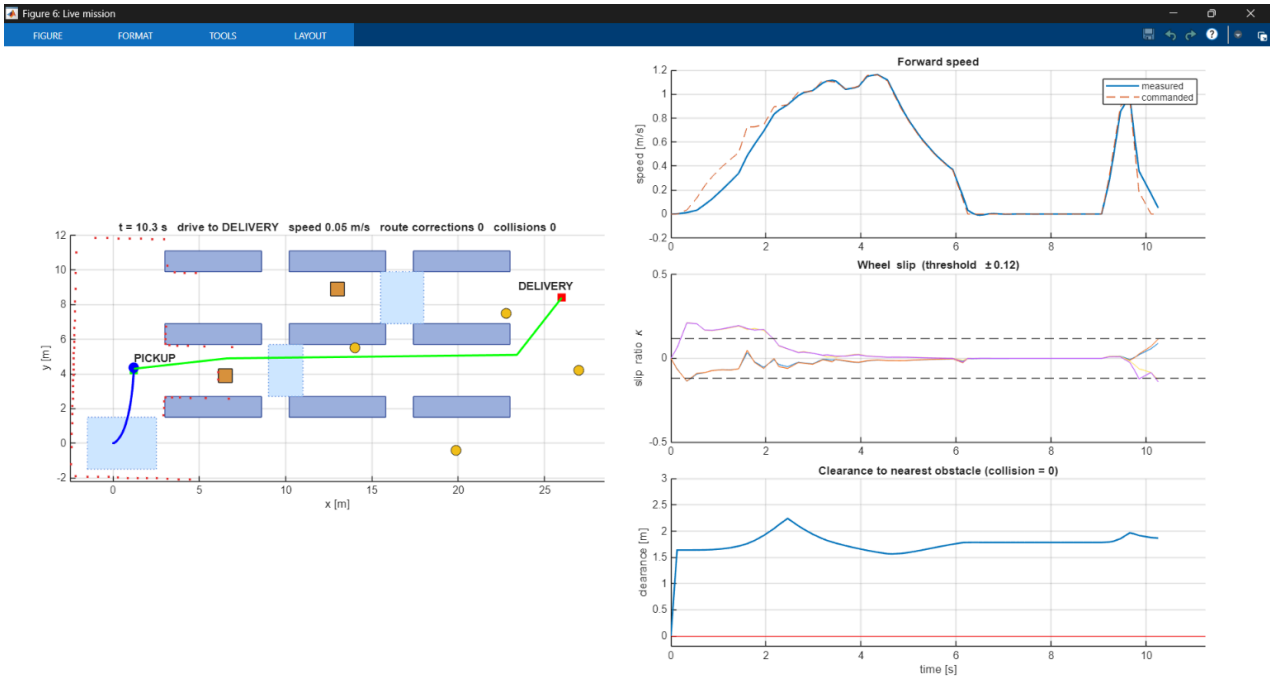


Fig. 27: Live figure of live_graphs.m during the mission (t = 10.3 s): map with the driven path (blue), the planned path (green), LiDAR hits (red dots) and the moving obstacles; forward speed, wheel slip ratios and clearance against time

- Map (left): racks, floor boxes, low-friction patches and the two stations; the title shows the time, the mission state, the speed, the number of route corrections and the number of collisions.
- Forward speed: measured against commanded; the robot has stopped for the 3 s loading (6.1 to 9.1 s) and is accelerating again.
- Wheel slip: the slip ratio of the four wheels with the ±0.12 detection lines; the oil patch at the start gives the largest slip in the first 2 s.
- Clearance: the distance from the robot to the nearest obstacle against the collision line at zero.

The live figure shows the same signals as Figs. 17, 18 and 19 and the map of Fig. 15, but as they happen. It is a viewing tool: the saved results and all numbers of this report come from `run_scout_warehouse_simulation`.

15. Mathematical Formulation of the Implementation

This section collects the equations that the MATLAB files implement, so that every number in the previous sections can be traced to a formula. The symbols follow the code: r is the wheel radius (0.085 m), b the track (0.612 m), Ts the control period (5 ms) and g = 9.81 m/s².

15.1 Frames and the footprint pose

The imported base_link frame is rotated by +90° about Z with respect to the driving frame base_footprint, and its origin is offset from the footprint by p_fb = (−0.10185, 0.16662, 0.2269) m (taken from the URDF). The controller therefore reconstructs the footprint pose from the base_link pose (p_b, R_b) as

```
R_wf = R_b * [0 1 0; -1 0 0; 0 0 1]
p_f  = p_b - R_wf * p_fb
psi  = atan2(R_wf(2,1), R_wf(1,1))
v_f  = v_b - w_b x (R_wf * p_fb)
v_x  = v_f . e_f ,    v_y = v_f . e_l ,    e_f = [cos psi; sin psi],  e_l = [-sin psi; cos psi]
```

The planar pose (x, y, ψ) is what the navigation receives; v_x, v_y and the yaw rate ω_z are used for the slip ratio. The sign vector sgn = (sgn_1…sgn_4) tells in which direction a positive joint rotation moves the robot forward; it is computed once in `scout_amr_params.m` from the URDF joint rotations (roll/pitch/yaw of each wheel joint and the joint axis), because the wheels on the left and the right side of the exported model have opposite axes.

15.2 Skid-steer kinematics

A skid-steer robot has no steering mechanism: the heading changes by driving the two sides at different speeds. With the requested speed v and yaw rate ω the wheel-speed targets of the left (w1, w2) and the right (w3, w4) wheels are

```
w_L = (v - b/2 * omega) / r        w_R = (v + b/2 * omega) / r
w_c = [w_L; w_L; w_R; w_R]
```

For v = 1.2 m/s and the largest yaw-rate request ω = 1.3 rad/s these give w_L = 9.4 rad/s and w_R = 18.8 rad/s, below the joint velocity limit of 25 rad/s of the URDF. Because the wheels cannot move sideways, the real turning behaviour differs from this ideal model and the wheels scrub sideways; this is one source of the slip ratio spikes in sharp turns that Section 10 describes.

15.3 Request shaping

The speed and yaw-rate requests of the navigation are not applied directly. The controller limits their rates:

```
a_lim = min(a_max, 0.85 * mu_est * g)            (slip management on)
v_cmd = v_prev + clip(v_ref - v_prev, -1.6*a_lim*Ts, +a_lim*Ts)
w_cmd = w_prev + clip(w_ref - w_prev, -4*Ts, +4*Ts)
```

so the robot may brake up to 1.6 times harder than it accelerates, and the yaw rate is slewed at 4 rad/s². With a_max = 2.5 m/s² and $\hat{\mu}$ = 0.9 the limit is 2.5 m/s² (the minimum of 2.5 and 7.5); with $\hat{\mu}$ = 0.10, as on the oily dock, it falls to 0.83 m/s².

15.4 Tyre-ground contact

Each wheel has one contact point directly below the hub centre. With the hub height z_h, the penetration depth, the vertical contact force and the friction force are

```
pen = r - z_h
F_z = max(0, k_c * pen - c_c * v_c,z)             k_c = 12000 N/m, c_c = 700 N s/m (only if
pen > 0)
F_t = - mu * F_z * v_s / sqrt(|v_s|^2 + v_0^2)    v_0 = 0.03 m/s, v_s = horizontal contact-
point velocity
T   = (p_c - p_O) x [F_t ; F_z]
```

The smoothed Coulomb law avoids the discontinuity of the sign function at zero sliding speed: for |v_s| much larger than v_0 the friction force approaches μ·F_z, and for small v_s it is proportional to v_s (slope μ·F_z/v_0), which keeps the solver stable. The friction coefficient μ is looked up in the table of patches: 0.15 on the oily dock, 0.30 on the two water spills and 0.90 elsewhere. The force and torque are then rotated into the wheel frame and applied to the wheel body by the Simulink block.

15.5 Slip ratio

The longitudinal slip ratio of wheel k is the normalised difference between the wheel's rolling speed and the ground speed at the wheel:

```
v_k   = v_x - omega_z * y_k                    (y_k = +0.30 m left, -0.30 m right)
kappa = (r*w_k - v_k) / max(|r*w_k|, |v_k|, 0.8 m/s)
```

A positive κ means that the wheel spins faster than the ground passes (acceleration slip) and a negative κ that it is slower (lock-up). The floor of 0.8 m/s in the denominator keeps κ finite at standstill; the price is that at very low speed the slip ratio understates the relative slip.

15.6 Wheel-speed control

For each wheel the torque demand is the sum of an acceleration feed-forward, a proportional term and an integral term:

```
tau_ff = m * a_cmd * r / 4                      m = 30 kg
e      = w_c - w_k
I      = clip(I + K_i * e * Ts, -tau_lim, +tau_lim)
tau    = clip(tau_ff + K_p * e + I, -tau_lim, +tau_lim)      K_p = 0.3, K_i = 8
```

The integrator is clamped to the current torque limit, which is a simple anti-windup. The feed-forward carries part of the load during acceleration: for a_cmd = 2.5 m/s² it is 30·2.5·0.085/4 = 1.6 N·m per wheel, while the peak torque

seen in the reference run is only 6.2 N·m (Table 7), far below the 17.5 N·m limit. The torque limit tau_lim is reduced by the slip management (Section 10.1):

```
s = (|kappa_k| - 0.12) / (0.40 - 0.12)
tau_lim,k = 17.5 * max(0.1, 1 - s)      if s > 0, else 17.5
```

15.7 LiDAR ray casting

Beam i of the n = 72 beams points at the angle a_i = ψ − π + (i−1)·2π/n. A ray with direction d = (cos a_i, sin a_i) is intersected with every axis-aligned rectangle by the slab method: with t_x1,2 = (x_min,max − x)/d_x and t_y1,2 = (y_min,max − y)/d_y, the ray hits the box if t_far = min(max(t_x), max(t_y)) ≥ max(t_near, 0), where t_near = max(min(t_x), min(t_y)); the range is t_near. For a circle of radius ρ around c the range is the smaller root t = −b − √(b² − c0), with b = (p − c)·d and c0 = |p − c|² − ρ². The nearest hit over all objects is rounded to 2 cm and limited to 8 m. One scan needs 72 × 19 intersection tests (17 rectangles and 2 circles), which is small enough to be run every 0.1 s inside the 5 ms control loop.

15.8 Occupancy grid and A* cost

The grid covers x from −2.5 to 28.5 m and y from −2.2 to 12.0 m with 0.2 m cells (155 × 71 = 11 005 cells). A cell is marked as static if its centre lies closer than 0.48 m to a rack or wall rectangle. The dynamic layer stores the time of the last LiDAR hit in a disc of about 2.2 cells radius around the hit cell, and a cell counts as blocked for 2 s after its last hit. A* uses the octile distance h = (Δx + Δy) + (√2 − 2)·min(Δx, Δy) as heuristic, which is exact on an obstacle-free 8-connected grid and therefore admissible, and it forbids diagonal moves that cut a blocked corner. The string-pulling smoother then keeps only the waypoints from which the next one is not visible along a free line, and the path is limited to 80 waypoints.

15.9 Path following and speed limit

Pure pursuit selects the target point at a look-ahead distance of 0.9 m along the path, starting at the point of the path nearest to the robot. The desired direction is the unit vector to the target plus 0.25 times the repulsion vector, which sums the returns closer than 0.9 m with the weight (0.9 − r)/0.9 in the direction opposite to the beam. With the heading error e the commands are

```
omega_ref = clip(1.8 * e, -1.3, +1.3)
v_ref     = min(v_max, sqrt(2 * 0.8 * (r_front - 0.55))) * max(0, 1 - |e|/1.2)
v_ref     = min(v_ref, max(0.15, 0.8 * d_goal))
r_front < 0.55                    ->  v_ref = 0                          (safety stop)
r_front < 0.68 and r_rear > 0.9   ->  v_ref = -0.4, omega_ref = 0        (back away)
|e| > 1.2 and r_front >= 0.55     ->  omega_ref = sign(e) * 1.0          (turn in place)
```

The expression √(2·0.8·(r_front − 0.55)) is the speed from which the robot can stop within the free distance beyond the 0.55 m margin at a deceleration of 0.8 m/s². The front sector is |angle| < 0.9 rad and the rear sector |angle − π| < 0.9 rad of the 72 beams. The term min(v, 0.8·d_goal) slows the robot down when it approaches the station, and the mission state advances when the distance to the goal is below 0.40 m.

15.10 The log vector

The controller returns a log vector of 44 values at every 5 ms step, which the run script converts to the plots and to the result file (Table 11).

Index	Signal	Index	Signal
1–2	footprint position x, y [m]	29	friction estimate $\hat{\mu}$
3	yaw angle ψ [rad]	30	slip detected flag
4–6	v_x, v_y [m/s], yaw rate ω_z [rad/s]	31	mission state (1 to 5)
7–8	commanded v_cmd, ω_cmd	32	ground-truth clearance [m]
9–12	wheel-speed targets w_c [rad/s]	33	done flag
13–16	measured wheel speeds [rad/s]	34	unused (0)
17–20	slip ratios κ_1 … κ_4	35	filtered longitudinal acceleration [m/s²]
21–24	wheel torques after limiting [N·m]	36	accumulated slip time [s]
25–28	torque limits [N·m]	37	navigation request v_ref [m/s]
38	route-correction counter	42	front range r_front [m]
39	closest LiDAR return [m]	43	stuck flag

40	in-collision flag	44	distance to goal [m]
41	collision counter		

16. Verification and Discussion of the Results

A simulation result is only useful if one knows what was checked. This section lists the checks that were done on the model and the reference run, and discusses what the numbers of Sections 9 and 10 do and do not show. All values are taken from the saved result file of the reference run.

16.1 Verification checklist

No.	Check	Expected	Observed	Result
1	URDF imports without error into `importrobot` and `smimport`	base_link, four wheel bodies, four revolute wheel joints and the sensor mounts	Imported; the tree is shown in Fig. 5 and Fig. 6	Pass
2	Chassis rests on the floor without sinking or floating	Height constant after the first second	Settles about 0.7 mm below its starting height; tilt within 0.6°	Pass
3	Repeatability	Same inputs give the same numbers	A repeated run gave identical numbers (deterministic world, no noise)	Pass
4	Mission completes	State 5 (done) reached before 120 s	Done at 49.6 s	Pass
5	No collision	Collision circle (r = 0.36 m) never overlaps an obstacle	0 collisions; minimum clearance 0.22 m	Pass
6	Speed limit respected	$v \le 1.2$ m/s	Top speed 1.20 m/s	Pass
7	Torque within the motor limit	$\lvert\tau\rvert \le 17.5$ N·m	Peak wheel torque 6.17 N·m	Pass
8	Speed tracking	Measured speed follows the command	RMS speed error 0.050 m/s	Pass
9	Slip is limited by the management	Peak slip lower than without management	0.48 against 0.64	Pass
10	Friction estimate follows the floor	Falls on the oil and water, recovers on dry concrete	Minimum 0.10; maximum 0.90	Pass
11	Re-planning around blockages	Route corrections when the planned path is blocked	3 corrections at 25.1, 30.2 and 37.6 s	Pass
12	Comparison with the real robot	Model agrees with measurements	Not done (no hardware test)	Not checked

16.2 Timeline of the reference run

Time [s]	Event
0	Start on the oily dock ($\mu = 0.15$); the slip management limits the acceleration and the wheel speeds
6.1	Arrival at the pickup station (distance to goal below 0.40 m); loading starts
9.1	Loading finished after the 3 s dwell; the robot starts towards the delivery station
≈ 17	The floor box in aisle 2 is detected by the LiDAR and the path bends around it
25.1	First route correction: worker 1 stands about 1 m ahead in aisle 2 and blocks the current path
≈ 27	The pallet truck, released when the robot passed x = 12 m, stops at x = 20.4 m and closes aisle 2
30.2	Second route correction: the path is blocked again by worker 1 in aisle 2
37.6	Third route correction in aisle 3 with worker 2 and the truck within 5 m
46.6	Arrival at the delivery station; unloading starts
49.6	Unloading finished; mission state done; the simulation stops

16.3 Discussion

Navigation. The robot covered 32.0 m in 49.6 s, of which 6 s were spent standing during loading and unloading, so the mean speed of 0.64 m/s refers to the whole mission including the waits. The straight-line distance between the two stations is about 25 m, so the driven route is longer than the shortest one because the robot has to pass through the cross-aisles and around the box and the people. The three route corrections show that the dynamic layer, the 0.5 s planning period and the retreat behaviour work together; the 4 s debounce prevents the counter from counting one blockage several times.

Safety margin. The closest approach between the collision circle and any obstacle was 0.22 m, and the closest LiDAR return was 0.58 m, which is above the 0.55 m safety-stop distance. These margins are valid for the collision

radius of 0.36 m, which is a circle around the 0.672 × 0.580 m chassis outline; they would shrink if the circle were replaced by the real footprint.

Slip management. The comparison in Table 8 shows a trade-off rather than a pure gain. With the management the peak slip ratio drops from 0.64 to 0.48 and the time with a slip ratio above 0.2 from 5.4 % to 2.1 %, but the mission is 4.1 s slower, the mean speed is lower (0.64 against 0.69 m/s), and the RMS speed error is larger (0.050 against 0.026 m/s) because the traction window deliberately lets the wheels lag the commanded speed on slippery floor. The energy is almost the same. Whether this trade-off is worthwhile depends on the application: for loads that can shift or floors that are often wet, the smoother wheel-ground behaviour can be worth the extra seconds.

Friction estimate. The estimator reduces $\hat{\mu}$ by 0.5 per second while any wheel slips and raises it by 0.15 per second otherwise. It reached 0.10, the lower bound, in the slippery zones, which means that the estimate is a conservative indicator of slip activity and not a measurement of the true friction coefficient; the true values are 0.15 (oil) and 0.30 (water). A low estimate also reduces the acceleration limit, so the robot is cautious for several seconds after it leaves a slippery zone.

Sensitivity. The behaviour was not tested under changes of the parameters (look-ahead distance, inflation radius, tyre stiffness, controller gains), under sensor noise or with other layouts. The gains Kp = 0.3 and Ki = 8 were chosen to be stable for the wheel inertia of 0.005 kg·m² at the 5 ms step, and the torque chatter in Fig. 19 shows that they were not tuned for smoothness. A systematic parameter study is listed among the future enhancements.

17. Limitations and Future Enhancements

17.1 Limitations

- The pose comes from the simulator: there is no localisation, SLAM or sensor fusion, and the IMU and camera are only mounted, not simulated, in the Simulink model.
- The LiDAR is analytic and noise-free; collision detection is analytic. There is no Simscape contact between the robot and obstacles.
- The moving obstacles follow scripted paths; they are partly aware of the robot.
- The tyre model is a simple point-contact model per wheel; it has no tyre deformation, rolling resistance model or suspension.
- Motors are ideal torque sources limited to 17.5 N·m; battery, motor electrical dynamics and communication delays are not modelled.
- Torque and normal-load signals are noisy, and the slip comparison is one run per case.
- No validation against the real Scout Mini was done.

17.2 Future enhancements

- Add noisy LiDAR, IMU and wheel odometry as Simulink signals and replace the true pose by an estimator (EKF or AMCL / SLAM).
- Add contact forces between the robot and the obstacles (Simscape contact or Spatial Contact Force) so that collisions are physical.
- Model the motors and the battery with Simscape Electrical, and the drive line with Simscape Driveline.
- Predict the motion of workers and other vehicles instead of reacting to each scan; add fleet traffic rules at the cross-aisles.
- Test more layouts, random obstacle placements and several runs per slip configuration; tune the controller to reduce torque chatter.
- Validate the model against measurements of the real Scout Mini (mass, wheel radius, speed limits, slip on real floors).

18. Conclusion

The AgileX Scout Mini was modelled in SolidWorks, exported to URDF and STL, imported into MATLAB with `importrobot` and into Simulink with `smimport`, and turned into a dynamic Simscape Multibody plant with a free chassis, torque-driven wheels and a custom tyre-ground friction model. A 3D warehouse with racks, goods, stations, spills, static boxes and four moving obstacles was built into the same multibody scene. The robot navigates autonomously with a LiDAR, an occupancy grid, A* planning and re-planning, pure pursuit and a mission state machine, while a slip-management layer limits wheel slip. In the reference run it delivered the load in 49.6 s with 3 route corrections and 0 collisions, and it handled a floor box, a crossing worker, an aisle closed by a pallet truck and a spill-covered floor. With slip management the peak slip ratio was 0.48 instead of 0.64, at the cost of about 4.1 s of mission time.

These results come from simulation with the true pose, analytic sensing and scripted moving obstacles on a single invented layout, so they show that the software chain works, not that the physical robot would behave the same. The next steps are localisation with noisy sensors, physical contact with obstacles, a motor and battery model, and hardware tests.

19. References

- The MathWorks, Inc. MATLAB, Simulink, Simscape Multibody and Robotics System Toolbox documentation, release R2025b (functions `smimport`, `importrobot`, `rigidBodyTree`, `ode23t`).
- Open Robotics. URDF specification (Unified Robot Description Format), ROS documentation.
- AgileX Robotics. SCOUT MINI user manual and product data sheet.
- P. E. Hart, N. J. Nilsson and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968.
- R. C. Coulter, “Implementation of the pure pursuit path tracking algorithm,” Technical Report CMU-RI-TR-92-01, Carnegie Mellon University Robotics Institute, 1992.
- H. Pacejka, Tyre and Vehicle Dynamics, 3rd ed., Butterworth-Heinemann, 2012 (slip ratio definition and friction models).
- R. Siegwart, I. R. Nourbakhsh and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011.
- S. Thrun, W. Burgard and D. Fox, Probabilistic Robotics, MIT Press, 2005 (occupancy grids, range sensing).
- International Organization for Standardization. ISO 3691-4: Industrial trucks – Safety requirements and verification – Part 4: Driverless industrial trucks and their systems.
- SolidWorks documentation, URDF export of assemblies (Assem2: chassis, chassis_top, W, up).

Appendix A. Complete Parameter Listing

All constants of the simulation are defined in `scout_amr_params.m`. The tables list them by group; a number changed in this file changes the whole simulation.

Symbol / field	Value	Meaning
P.r	0.085 m	wheel radius used for rolling and for the contact point
P.hubOff	0.062 m	hub centre offset from the wheel frame along the axle
P.track	0.612 m	effective track width
P.yWheel	[0.30 0.30 −0.30 −0.30] m	lateral offsets of w1 (front left), w2 (rear left), w3 (front right), w4 (rear right)
P.tmax	17.5 N·m	peak torque of one in-hub motor
P.p_fb	[−0.10185, 0.16662, 0.2269] m	base_link origin in base_footprint (URDF)
P.rRobot	0.38 m	avoidance radius of the robot
P.rCol	0.36 m	collision radius used by the collision monitor
P.kc, P.cc	12 000 N/m, 700 N·s/m	vertical contact stiffness and damping per wheel

P.v0	0.03 m/s	Coulomb-friction smoothing velocity
P.mu0	0.90	friction coefficient of dry concrete
P.Ts	0.005 s	control period
P.vmax, P.amax	1.2 m/s, 2.5 m/s²	speed limit and nominal acceleration demand
P.Kp, P.Ki	0.3, 8.0	wheel-speed PI gains
P.mass	30 kg	robot mass in the acceleration feed-forward
P.vSlipFloor	0.8 m/s	speed floor in the slip-ratio denominator
P.k_th, P.k_tgt, P.k_max	0.12, 0.08, 0.40	slip detection threshold, regulated slip ratio, slip ratio at which the torque is cut to the minimum
P.d0	0.9 m	obstacle influence distance of the repulsion
P.tStop	120 s	simulation stop time (the mission ends earlier)

Symbol / field	Value	Meaning
P.racks	9 rectangles: x 3.0–8.6, 10.2–15.8, 17.4–23.0; y 1.5–2.7, 5.7–6.9, 9.9–11.1	three rack rows with three segments each; the gaps at x = 8.6–10.2 and 15.8–17.4 are the cross-aisles
P.walls	x −2.5 … 28.5, y −2.2 … 12.0	four wall rectangles, 0.2 m thick
P.boxes	[6.1 6.9 3.5 4.3], [12.6 13.4 8.5 9.3]	floor boxes in aisles 2 and 3, unknown to the planner
P.patches	0.15 (x −1.5…2.5, y −1.5…1.5); 0.30 (x 9…11, y 2.7…5.7); 0.30 (x 15.5…18, y 6.9…9.9)	oil on the loading dock and two water spills
P.pickup, P.deliver	(1.2, 4.2) m, (26.0, 8.4) m	pickup and delivery stations
P.dwell	3.0 s	loading and unloading time
P.mover.size	0.6×0.6, 0.6×0.6, 0.9×0.6, 1.4×2.6 m	worker 1, worker 2, second AMR, pallet truck
P.mover.truckX0/X1/V/TrigX	27.0 m, 20.4 m, 1.0 m/s, 12.0 m	truck start, stop position, speed and the robot x at which it is released
P.lidar.n, P.lidar.rmax	72, 8.0 m	number of beams and maximum range
P.grid.res	0.2 m	cell size (155 × 71 cells, origin (−2.5, −2.2) m)
P.grid.inflate	0.48 m	inflation of static obstacles
P.grid.dynHold	2.0 s	time a LiDAR hit keeps a cell blocked
P.grid.replanT	0.5 s	planning period

Appendix B. Glossary

Term	Meaning
AMR	Autonomous mobile robot: plans its own route and avoids obstacles, unlike an AGV that follows a fixed track
AGV	Automated guided vehicle that follows wires, tape or magnets
A*	Best-first graph search that expands the node with the lowest cost-so-far plus heuristic
Occupancy grid	Map that divides the floor into cells marked free or occupied
Inflation	Growing the obstacles by the robot radius plus a margin so that the robot can be treated as a point
LiDAR	Light detection and ranging; here a planar scanner that measures the distance to the nearest obstacle in each beam direction
Pure pursuit	Path-following method that steers towards a point at a fixed look-ahead distance on the path
Skid steer	Steering by driving the wheels on the two sides at different speeds, with no steered axle
Slip ratio κ	Normalised difference between wheel rolling speed and ground speed at the wheel
Traction control	Limiting wheel torque and speed so that the slip ratio stays near a target
PI controller	Controller with a proportional and an integral term
URDF	Unified Robot Description Format, an XML description of links, joints and meshes
STL	Triangle-mesh file format used for the link shapes
Simscape Multibody	Simulink library for 3D rigid-body dynamics
ode23t	Variable-step solver for moderately stiff problems, used for the multibody model
Dwell time	Time the robot stands at a station for loading or unloading
ISO 3691-4	Safety standard for driverless industrial trucks

Appendix C. Source Code of the Main Files

The listings are the files as used for the results of this report. They are shown to document the implementation in full. The two files that connect the Simulink plant with the navigation are listed: the controller, which is called from a

MATLAB Function block every 5 ms, and the tyre model, which is called once per wheel. The navigation files are described in Sections 11 and 15 and are in the repository.

C.1 scout_amr_controller.m (navigation call, wheel-speed control, slip management)

```
function [tau_j, lg, mv, rng, pth] = scout_amr_controller(pb, Rb, vb, wb, wj, tc_on)
%SCOUT_AMR_CONTROLLER  Mission navigation, wheel-speed control and traction (slip) management.
%  pb,Rb,vb,wb : base_link pose / velocity from the multibody model (world frame)
%  wj          : joint speeds of the four wheel revolute joints [rad/s]
%  tc_on       : 1 = slip management active, 0 = plain wheel-speed PI (for comparison)
P = scout_amr_params();
dt = P.Ts;
persistent init vc_prev wc_prev I mu_est vx_prev ax_f slipTimer
if isempty(init)
    init = true; vc_prev = 0; wc_prev = 0; I = zeros(4,1);
    mu_est = P.mu0; vx_prev = 0; ax_f = 0; slipTimer = 0;
end

% ---- Pose of base_footprint from base_link ------------------------------
Rwf = Rb * [0 1 0; -1 0 0; 0 0 1];
pf  = pb(:) - Rwf * P.p_fb;
psi = atan2(Rwf(2,1), Rwf(1,1));
vf  = vb(:) - cross(wb(:), Rwf * P.p_fb);
ef  = [cos(psi); sin(psi)];  el = [-sin(psi); cos(psi)];
vx  = vf(1:2).' * ef;  vy = vf(1:2).' * el;  wz = wb(3);
pos = pf(1:2);

% ---- Warehouse autonomy: mission, LiDAR, A* replanning, path following ------------------
[vref, wref, mv, rng, pth, info] = scout_wh_nav(pos(1), pos(2), psi, dt);
mv = reshape(mv.', 8, 1);          % [x1; y1; x2; y2; ...] for the 3D movers
done = info(9) > 0.5;
err = 0;

% ---- Friction-aware acceleration limit ----------------------------------
if tc_on > 0.5
    alim = min(P.amax, 0.85 * mu_est * 9.81);
else
    alim = P.amax;
end
vcmd = vc_prev + max(-1.6*alim*dt, min(alim*dt, vref - vc_prev));
acmd = (vcmd - vc_prev) / dt;
wcmd = wc_prev + max(-4*dt, min(4*dt, wref - wc_prev));
vc_prev = vcmd; wc_prev = wcmd;

wL = (vcmd - 0.5*P.track*wcmd) / P.r;
wR = (vcmd + 0.5*P.track*wcmd) / P.r;
wc = [wL; wL; wR; wR];

% ---- Slip estimation from wheel speed vs. ground speed ------------------
wfwd = P.sgn(:) .* wj(:);
vk   = vx - wz * P.yWheel(:);              % ground speed at each wheel's lateral position
```

```
den  = max(max(abs(P.r*wfwd), abs(vk)), P.vSlipFloor);   % low-speed regularisation of the
slip ratio
kappa = (P.r*wfwd - vk) ./ den;
slip = any(abs(kappa) > P.k_th);

% ---- Traction management ------------------------------------------------
tauLim = P.tmax * ones(4,1);
if tc_on > 0.5
    wgl = vk / P.r;                          % wheel speed that matches the ground
    dW  = (P.k_tgt*abs(vk) + 0.12) / P.r;    % allowed wheel-speed deviation (target slip)
    wc  = max(wgl - dW, min(wgl + dW, wc));  % anti-spin / anti-lock speed window
    for k = 1:4
        s = (abs(kappa(k)) - P.k_th) / (P.k_max - P.k_th);
        if s > 0, tauLim(k) = P.tmax * max(0.1, 1 - s); end
    end
    if slip, mu_est = max(0.10, mu_est - 0.5*dt); else, mu_est = min(P.mu0, mu_est + 0.15*dt);
end
end
if slip, slipTimer = slipTimer + dt; end

% ---- Wheel-speed PI -----------------------------------------------------
e = wc - wfwd;
I = max(-tauLim, min(tauLim, I + P.Ki*e*dt));
tau_ff = P.mass * acmd * P.r / 4;             % acceleration feed-forward
tau_d = max(-tauLim, min(tauLim, tau_ff + P.Kp*e + I));
tau_j = P.sgn(:) .* tau_d;
if tc_on < 0, tau_j(:) = 0; end   % diagnostic: motors off

ax_raw = (vx - vx_prev)/dt; vx_prev = vx; ax_f = 0.99*ax_f + 0.01*ax_raw;

lg = [pf(1); pf(2); psi; vx; vy; wz; vcmd; wcmd; wc; wfwd; kappa; tau_d; tauLim; ...
      mu_est; double(slip); info(1); info(5); double(done); err; ax_f; slipTimer; vref; ...
      info(2); info(3); info(4); info(8); info(6); info(7); info(10)];
end
```

C.2 scout_amr_tires.m (tyre-ground contact model)

```
function out = scout_amr_tires(pO, vO, R, w, Rb, k)
%SCOUT_AMR_TIRES  Compliant ground contact + friction (slip) model for one wheel.
%  k = wheel index 1..4.  Inputs (world frame): wheel-frame origin position/velocity, wheel
rotation matrix,
%  wheel angular velocity, chassis rotation matrix.  Output (11x1):
%  [F_local(3); T_local(3); Fz; Flong; Flat; kappa; mu] -- force/torque expressed in the wheel
frame.
P  = scout_amr_params();
pO = pO(:); vO = vO(:); w = w(:);
% In the imported Simscape model the joint frame's Z axis is the wheel axle; point it outboard
a   = R(:,3);
if sign(P.yWheel(k)) * a(2) < 0, a = -a; end
ph  = pO + P.hubOff * a;                    % hub centre
pc  = ph - [0; 0; P.r];                     % contact point directly below the hub
vc  = vO + cross(w, pc - pO);               % contact-point velocity

pen = P.r - ph(3);
Fz  = 0;
if pen > 0
    Fz = max(0, P.kc * pen - P.cc * vc(3));
end

mu = P.mu0;
for i = 1:size(P.patches,1)
    q = P.patches(i,:);
    if pc(1) >= q(1) && pc(1) <= q(2) && pc(2) >= q(3) && pc(2) <= q(4)
        mu = q(5);
    end
end

vs = vc(1:2);
Ft = -mu * Fz * vs / sqrt(vs.'*vs + P.v0^2);     % smoothed Coulomb friction (sliding of
contact patch)
Fw = [Ft; Fz];
Tw = cross(pc - pO, Fw);

Rwf = Rb * [0 1 0; -1 0 0; 0 0 1];               % footprint rotation (base_link yaw = +90
deg)
psi = atan2(Rwf(2,1), Rwf(1,1));
ef  = [cos(psi); sin(psi)]; el = [-sin(psi); cos(psi)];
wa  = w.' * a;                                    % absolute spin about the axle
vh  = vO + cross(w, ph - pO);                     % hub velocity
vwheel = P.r * wa * (cross(a, [0;0;1]).' * [ef; 0]);   % ground speed implied by wheel spin
vx  = vh(1:2).' * ef;
den = max([abs(vwheel), abs(vx), P.vSlipFloor]);
kappa = (vwheel - vx) / den;                      % >0 wheel spin, <0 lock-up

out = [R.' * Fw; R.' * Tw; Fz; Ft.' * ef; Ft.' * el; kappa; mu];
end
```